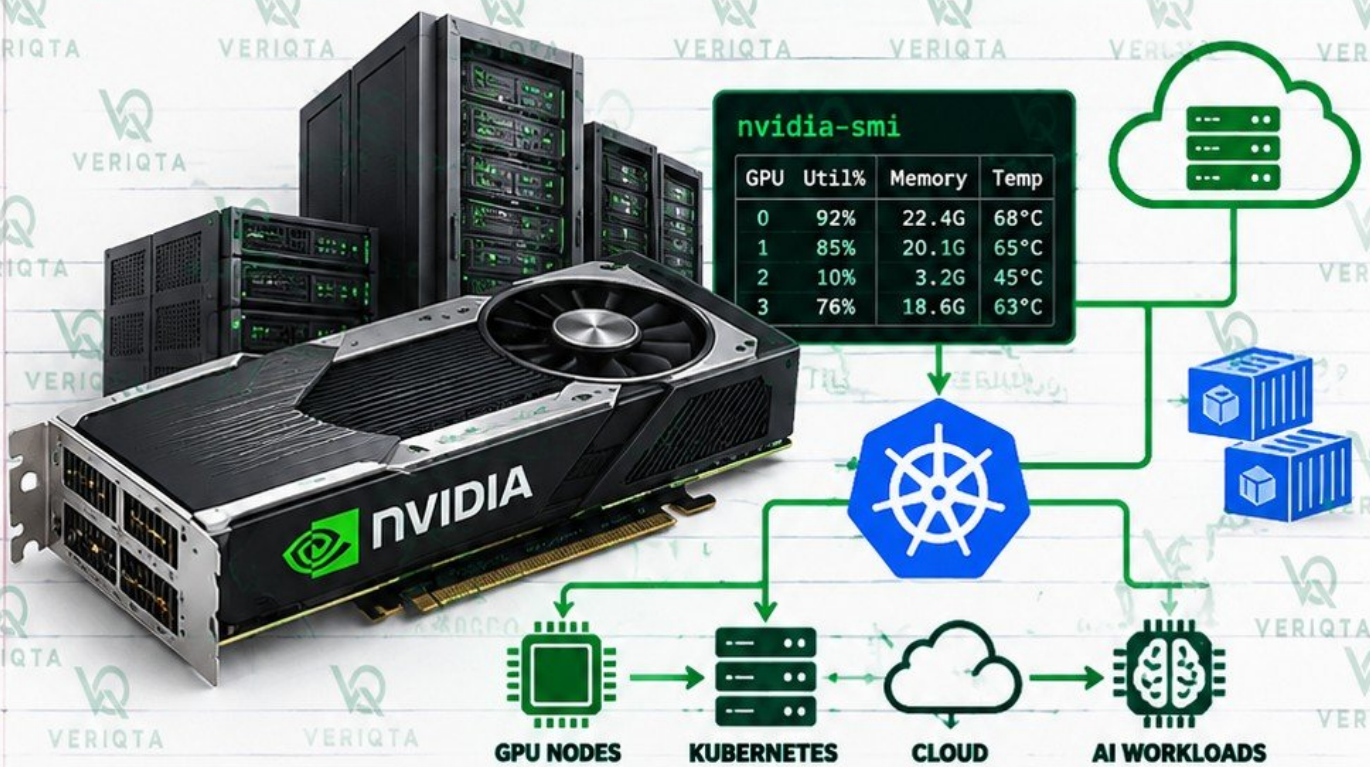


GPU

INFRASTRUCTURE

FOR

PLATFORM ENGINEERS




**ARCHITECTURE
DESIGN**


**DEPLOYMENT
OPERATIONS**


**PERFORMANCE
OPTIMIZATION**


**SECURITY
AND COMPLIANCE**


**RELIABILITY
AND SCALING**



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

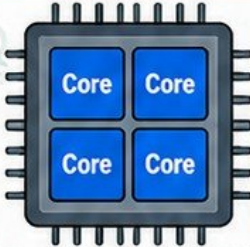


@veriqta

CPU vs GPU execution, SMs, memory hierarchy, PCIe, NVLink, HBM, and infrastructure implications.

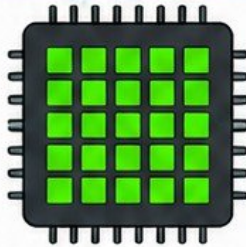
CPU VS GPU EXECUTION

CPU (General Purpose)



- Few powerful cores
- Optimized for low latency
- Excellent for sequential and complex logic
- Large cache (L1/L2/L3)
- Handles OS, networking, control plane, system tasks

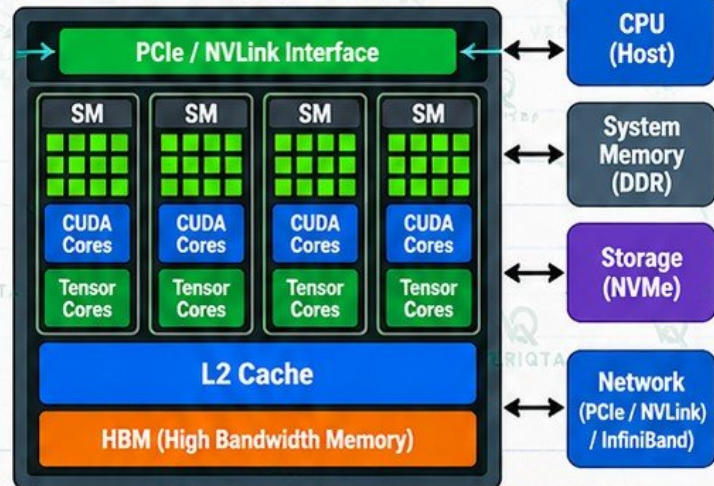
GPU (Accelerated Compute)



- Thousands of smaller cores (CUDA cores inside SMs)
- Optimized for high throughput
- Excellent for parallel workloads
- High memory bandwidth (HBM)
- Ideal for AI training, inference, simulation, video processing
- Works with CPU, not replace it

GPU ARCHITECTURE OVERVIEW

NVIDIA GPU (Simplified View)



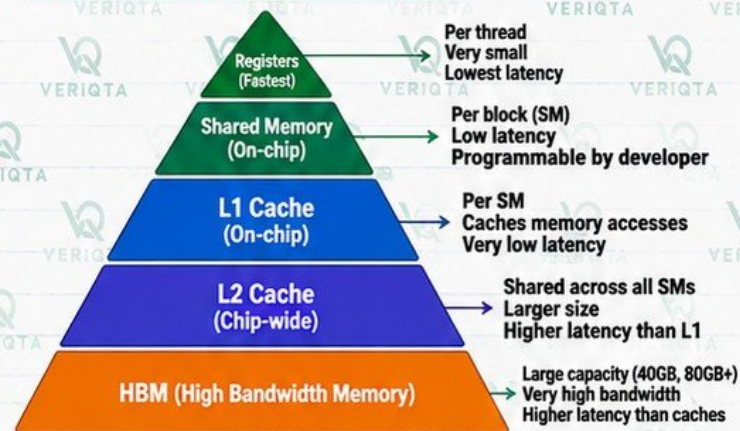
STREAMING MULTIPROCESSOR (SM)

SM (Streaming Multiprocessor)



- SM is the core building block of a GPU
- Each SM contains hundreds of CUDA cores
- Scheduler manages thousands of threads (warps)
- Tensor cores accelerate matrix operations (AI/ML)
- Shared memory is low latency and used for inter-thread data
- Many SMs run in parallel across the GPU

GPU MEMORY HIERARCHY



HBM (High Bandwidth Memory) gives GPUs very high memory bandwidth, which is critical for AI training and large model inference.

KEY INTERCONNECTS

PCIe (CPU ↔ GPU)



- Standard interconnect
- Common in single GPU servers
- PCIe 4.0 / 5.0
- Lower bandwidth than NVLink
- Used for most deployments

NVLink (GPU ↔ GPU)



- High-speed GPU-to-GPU link
- Much higher bandwidth than PCIe
- Used in multi-GPU servers
- Reduces communication latency during training
- Essential for large models

HBM (GPU Memory)



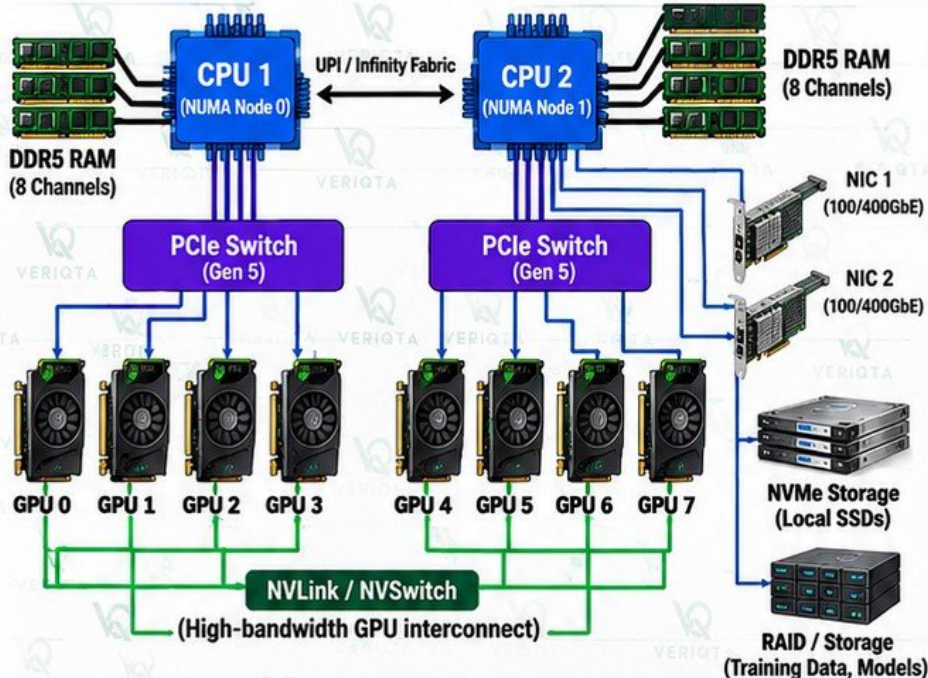
- On-GPU high bandwidth memory
- Very high throughput (TB/s)
- Enables large model training and inference
- Different versions: HBM2, HBM3
- Capacity: 40GB, 80GB, 96GB+

INFRASTRUCTURE IMPLICATIONS

- AI workloads require GPU-optimized servers with sufficient PCIe lanes, power and cooling.
- Multi-GPU training needs NVLink or high-speed networking (InfiniBand).
- HBM size and bandwidth directly impact model size and training speed.
- GPU scheduling, isolation and observability are essential in shared environments.
- Data pipelines must be designed to avoid data-transfer bottlenecks (CPU ↔ GPU).

GPU topology, NUMA, PCIe lanes, CPU, RAM, NICs, storage, and hardware bottlenecks.

PRODUCTION GPU SERVER ARCHITECTURE (EXAMPLE: 8 x GPU)

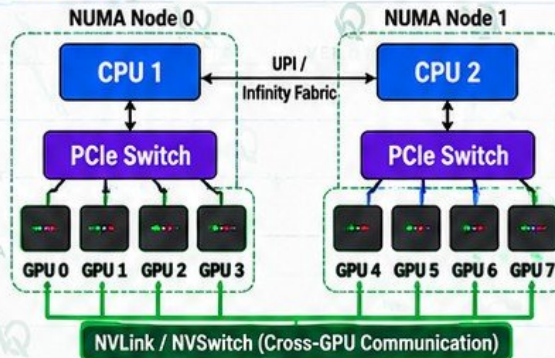


KEY COMPONENTS AND ROLE

CPU	- Coordinates GPU workloads - Handles OS, networking, storage - Feeds data to GPUs
GPU	- High parallel compute (CUDA cores) - Uses high-bandwidth memory (HBM) - Connected via PCIe and NVLink
RAM	- System memory for CPU workloads - Caches data before GPU transfer - Can be large (512GB – 2TB+)
PCIe	- Connects CPUs, GPUs, NICs, storage - Gen 4 or Gen 5 in modern servers - Bandwidth critical for data transfer
NVLink	- High-speed GPU-to-GPU link - Much higher bandwidth than PCIe - Reduces multi-GPU communication latency
NIC	- High-speed networking (100/400GbE) - Required for distributed training - Uses RDMA (RoCE/InfiniBand)
Storage	- High-throughput NVMe SSDs - Stores datasets, models, checkpoints - High IOPS and bandwidth required

GPU TOPOLOGY EXAMPLE (8 x GPU)

- GPUs are connected to specific CPUs through PCIe switches.
- GPUs on the same CPU/PCIe switch have lower latency.
- NVLink or NVSwitch provides high-bandwidth GPU-to-GPU communication.
- Topology affects training performance and multi-GPU scaling.



NUMA AND GPU PLACEMENT

- Each CPU has its own memory (NUMA node).
- GPUs are usually attached to a specific NUMA node via PCIe.
- Accessing a GPU from a remote NUMA node increases latency.
- Place CPU workers and containers on the same NUMA node as the GPU.
- Check NUMA topology using: `numactl --hardware`

REAL WORLD IMPACT

- Wrong NUMA placement can reduce GPU utilization and training performance.
- High cross-NUMA memory access increases latency and CPU overhead.

COMMON HARDWARE BOTTLENECKS

Bottleneck	Impact
PCIe bandwidth	Slows data transfer between CPU and GPU
Insufficient RAM	Data loading becomes a bottleneck
Slow storage	Training data cannot be read fast enough
Network bandwidth	Limits multi-node training performance
NUMA misplacement	Higher latency and lower GPU utilization
Thermal limits	GPU throttling reduces performance
Inadequate power and cooling	System instability and GPU shutdowns

USEFUL COMMANDS (LINUX)

<code>lspci grep -i nvidia</code>	Check GPU PCIe devices
<code>nvidia-smi topo -m</code>	Show GPU topology and links
<code>numactl --hardware</code>	View NUMA nodes and CPUs
<code>nvidia-smi -q</code>	Detailed GPU information
<code>nvidia-smi topo -p2p w</code>	Check peer-to-peer access
<code>lscpu</code>	CPU and NUMA information
<code>lsblk</code>	Check storage devices
<code>ethtool <interface></code>	Check NIC information

PRODUCTION CONSIDERATIONS

- Choose servers with sufficient PCIe lanes (Gen 4 or Gen 5).
- Use NVLink/NVSwitch for multi-GPU workloads.
- Ensure enough RAM for data preprocessing and caching.
- Use high-speed NVMe storage for datasets and checkpoints.
- Use 100/400GbE or InfiniBand for distributed training.
- Plan power and cooling for high GPU utilization.
- Monitor hardware health (temperature, power, ECC errors).



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

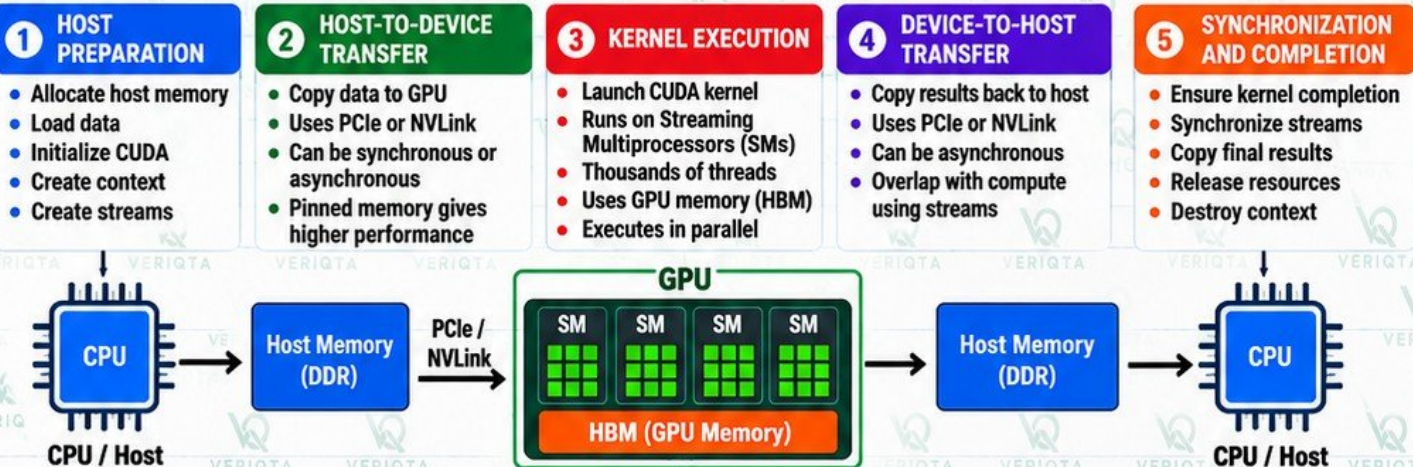


@veriqta

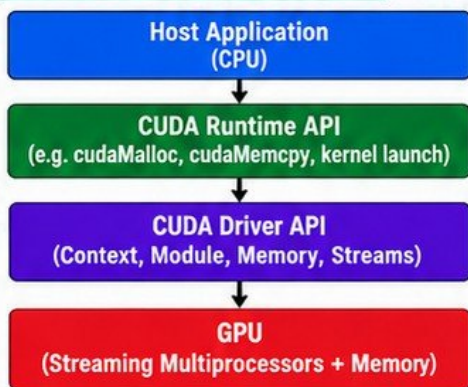
03 GPU WORKLOAD EXECUTION LIFECYCLE

Host-to-device transfer, CUDA contexts, kernels, streams, synchronization, and execution latency.

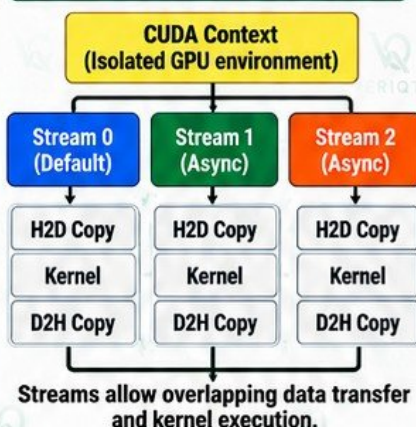
END-TO-END GPU WORKLOAD FLOW



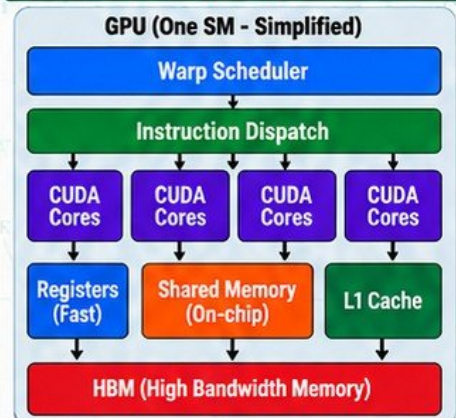
CUDA EXECUTION MODEL



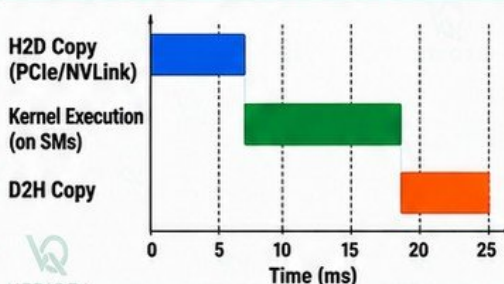
CUDA CONTEXT AND STREAMS



KERNEL EXECUTION INSIDE AN SM



SIMPLIFIED LATENCY TIMELINE



EXECUTION LIFECYCLE EXAMPLE (PYTHON - PYCUDA)

```
import pycuda.driver as cuda
import pycuda.autoninit
import numpy as np

# 1. Allocate host memory
h_data = np.random.rand(1<<20).astype(np.float32)
# 2. Allocate device memory
d_data = cuda.mem_alloc(h_data.nbytes)
# 3. Copy data to device (H2D)
cuda.memcpy_htod(d_data, h_data)
# 4. Launch kernel (example)
# kernel(d_data, block=(256,1,1), grid=(4096,1,1), stream=stream)
# 5. Copy result back (D2H)
cuda.memcpy_dtoh(h_data, d_data)
# 6. Synchronize
cuda.Context.synchronize()
```

KEY POINTS

- CUDA context isolates resources per process.
- Streams enable concurrent execution (copy + compute).
- Pinned memory improves transfer speed.
- Kernels run on thousands of threads inside SMs.
- Use asynchronous operations to overlap transfers and compute.
- Synchronize only when necessary to avoid idle time.

COMMON BOTTLENECKS

Bottleneck	Impact
PCIe bandwidth	Slower host-device transfers
Unpinned memory	Lower transfer throughput
Small transfer size	High overhead per transfer
Kernel launch overhead	High latency for many small kernels
Memory bandwidth (HBM)	Limits large model performance
Stream synchronization	Causes idle GPU time
CPU-side data preprocessing	GPU underutilization

OPTIMIZATION TIPS

- Use pinned memory for faster transfers
- Overlap H2D/D2H with kernel execution using streams
- Batch small operations to reduce launch overhead
- Keep GPU busy with larger workloads
- Minimize synchronization
- Use NVLink for multi-GPU communication
- Optimize memory access patterns for higher bandwidth
- Profile with tools like NVIDIA Nsight Systems and Nsight Compute

REAL-WORLD SCENARIO

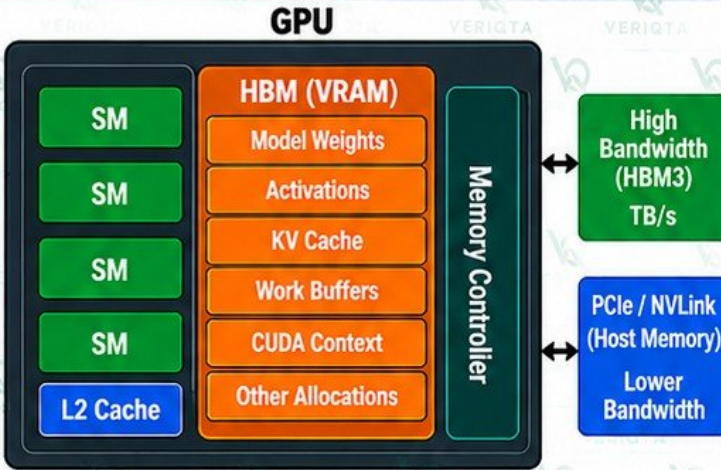
During large model inference, the GPU utilization is low (30%) even though the application is running. Investigation shows:

- Data loading and preprocessing on CPU is the bottleneck.
- Small batch size causing frequent transfers.
- Synchronous copies blocking kernel execution.
- Fix: use larger batches, pinned memory, and asynchronous streams to overlap transfers and compute.

04 GPU MEMORY MANAGEMENT UNDER PRESSURE

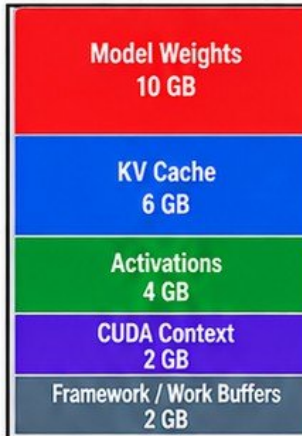
VRAM allocation, fragmentation, OOM, memory bandwidth, and shared GPU contention.

GPU MEMORY OVERVIEW



VRAM ALLOCATION IN A REAL WORKLOAD

GPU VRAM (24 GB Example)



Notes

- Model weights consume the largest portion.
- KV cache grows with concurrent requests.
- Activations depend on batch size and sequence length.
- CUDA context and framework buffers are always allocated.
- Insufficient VRAM leads to OOM errors or severe performance issues.

MEMORY FRAGMENTATION

Before (Contiguous Free Memory)



After (Fragmented Memory)



- Fragmentation happens when many allocations and deallocations create small free blocks.
- A large allocation may fail even if total free memory is sufficient.

SHARED GPU MEMORY CONTENTION

Multiple Workloads on One GPU



GPU VRAM (24 GB)

- Shared GPUs can improve utilization but increase contention.
- Memory spikes from one workload can cause OOM for others.
- Use MIG, time-slicing or strict limits for isolation.
- Monitor per-process memory usage and set requests/limits.

MEMORY BANDWIDTH AND PERFORMANCE

GPU	Memory Type	Bandwidth	Impact on Workloads
T4	GDDR6	320 GB/s	Good for inference
A10	GDDR6	600 GB/s	Inference and small training
A100	HBM2	1.6 TB/s	Large model training
H100	HBM3	3.35 TB/s	Large models, high throughput

- Higher memory bandwidth reduces memory bottlenecks.
- Memory-bound workloads (large models, embeddings) benefit the most.
- Bandwidth matters for model weights, activations and KV cache access.

COMMON GPU MEMORY ISSUES

Issue	Symptoms	Likely Cause	Fix / Mitigation
OOM (Out of Memory)	CUDA OOM error process killed	VRAM exceeded	Reduce batch size, use quantization, free memory, use MIG
Fragmentation	Allocation fails despite free memory	Many small allocations	Restart process, use memory pools
High memory usage	Slow performance	Large model or big batch size	Optimize model, reduce batch size
Memory leak	Memory usage keeps increasing	Framework or application issue	Profile and fix code, restart workers
Contention	Unstable performance	Multiple workloads sharing GPU	Use MIG, set limits, isolate workloads

USEFUL COMMANDS

```
# GPU memory usage
nvidia-smi
# Real-time monitoring
nvidia-smi dmon
# Per-process memory usage
nvidia-smi pmon -c 1
# Show GPU topology and memory
nvidia-smi topo -m
# Check CUDA processes
lsuf /dev/nvidia*
```

BEST PRACTICES

- Set appropriate memory requests and limits.
- Use MIG for multi-tenant workloads.
- Monitor memory usage and fragmentation.
- Use memory-efficient model formats (FP16, BF16, INT8).
- Optimize batch size and sequence length.
- Release unused memory in long-running processes.
- Use persistent processes to avoid repeated allocations.
- Monitor memory bandwidth and GPU utilization.

REAL WORLD LESSON

- High GPU utilization does not always mean good performance. If memory bandwidth is saturated, GPU can still be slow.
- Long running inference services can suffer memory fragmentation. Periodic restarts or memory pools can help.
- On shared GPUs, one misconfigured workload can cause OOM for everything.



@veriqta



@veriqta



@veriqta



@veriqta



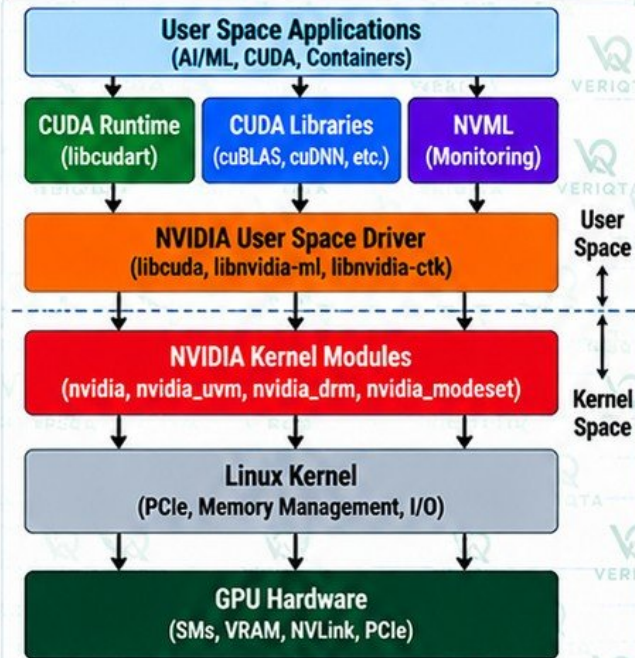
@veriqta



@veriqta

04

1. DRIVER ARCHITECTURE OVERVIEW



2. KEY NVIDIA KERNEL MODULES

Module	Purpose	Notes
<code>nvidia</code>	Core GPU driver	Handles GPU device control, memory, scheduling
<code>nvidia_uvm</code>	Unified memory module	Manages CUDA Unified Memory between CPU and GPU
<code>nvidia_drm</code>	DRM interface	Required for display and modesetting (even on headless servers)
<code>nvidia_modeset</code>	Modesetting component	Manages display modes
<code>nvidia_peermem</code>	RDMA support	Used for GPUDirect RDMA
<code>nvidia_fs</code>	Filesystem interface	Optional, for GDS (GPU Direct Storage)

3. NVIDIA DEVICE FILES

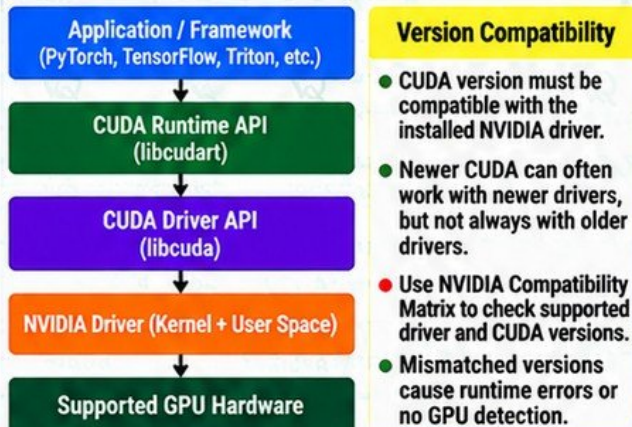
```
$ ls -l /dev/nvidia*
```

```
crw-rw-rw- 1 root root 195, 0 /dev/nvidia0
crw-rw-rw- 1 root root 195, 1 /dev/nvidia1
crw-rw-rw- 1 root root 195, 255 /dev/nvidiaactl
crw-rw-rw- 1 root root 511, 0 /dev/nvidia-uvm
crw-rw-rw- 1 root root 511, 1 /dev/nvidia-uvm-tools
crw-rw-rw- 1 root root 506, 0 /dev/nvidia-modeset
```

Important Files

- `/dev/nvidia0..N`
- GPU device files
- `/dev/nvidiaactl`
- Control interface
- `/dev/nvidia-uvm`
- Unified memory
- `/dev/nvidia-modeset`
- Modesetting interface

4. CUDA COMPATIBILITY STACK



5. USEFUL COMMANDS

```
# Check GPU and driver version
nvidia-smi

# Check loaded kernel modules
lsmod | grep nvidia

# Show driver details
modinfo nvidia

# Check GPU device files
ls -l /dev/nvidia*

# Check CUDA version (if installed)
nvcc --version

# Check NVML information
nvidia-smi -q

# Check kernel logs for NVIDIA errors
dmesg | grep -i nvidia

# Check driver and CUDA compatibility
nvidia-smi --query-gpu=name,driver_version,cuda_version --format=table
```

Check NVML from Python

```
import pynvml
pynvml.nvmlInit()
count = pynvml.nvmlDeviceGetCount()
for i in range(count):
    handle = pynvml.nvmlDeviceGetHandleByIndex(i)
    name = pynvml.nvmlDeviceGetName(handle)
    mem = pynvml.nvmlDeviceGetMemoryInfo(handle)
    print(f"GPU {i}: {name}")
    print(f"Total: {mem.total/(1024**3)} GB")
    print(f"Used : {mem.used/(1024**3)} GB")
    print(f"Free : {mem.free/(1024**3)} GB")
```

Check Kernel Logs for GPU Errors

```
dmesg | grep -iE "nvidia|NVRM|Xid"
```

6. COMMON DRIVER FAILURES

Failure	Symptoms	Likely Cause	Fix / Mitigation
Driver not loaded	<code>nvidia-smi</code> fails	Module not loaded or incompatible kernel	Install correct driver for kernel version
GPU not detected	No devices in <code>nvidia-smi</code>	PCIe issue, driver failure, BIOS setting	Check lspci, BIOS, reinstall driver
Xid errors	GPU resets, application crash	Hardware fault, memory error, thermal issue	Check <code>dmesg</code> , Xid code, replace GPU if persistent
CUDA version mismatch	Runtime errors	Incompatible CUDA and driver	Use supported version matrix

7. REAL WORLD EXAMPLE (XID ERROR)

```
$ dmesg | grep -i xid
```

```
[ 1234.567] NVRM: Xid (0000:65:00.0): 43, GPU has fallen off the bus.
[ 1234.567] NVRM: Xid (0000:65:00.0): 79, GPU has fallen off the bus.
```

Analysis

- Xid 43 or 79 indicates a serious hardware or PCIe communication issue.
- Can be caused by power, overheating, faulty GPU, or PCIe connectivity problems.
- Check server power, cooling, PCIe slot, firmware, and consider GPU replacement.

8. BEST PRACTICES

- Use the correct NVIDIA driver for your kernel and CUDA version.
- Keep drivers updated and tested in staging first.
- Monitor GPU health using NVML and DCGM.
- Enable persistence mode for stable device initialization.
- Check kernel logs regularly for Xid errors.
- Validate GPU detection after kernel updates.
- Use supported driver and CUDA versions from NVIDIA matrix.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

04

nvidia-smi, driver initialization, PCIe visibility, Xid errors, kernel logs, and fault isolation.

1. CHECK GPU STATUS WITH NVIDIA-SMI

\$ nvidia-smi

NVIDIA-SMI 550.54.14				Driver Version: 550.54.14				CUDA Version: 12.4			
GPU		Name		Persistence-M		Bus-Id		Disp.A		Volatile Uncorr. ECC	
Fan		Temp		Perf		Pwr:Usage/Cap		Memory-Usage		GPU-Util Compute M.	
=====											
0		NVIDIA A100-SXM4-40GB		On		00000000:17:00.0		Off		0	
N/A		33C		P0		67W / 400W		1024MiB / 40960MiB		12% Default	

KEY FIELDS

Driver Version	Installed NVIDIA driver version
CUDA Version	CUDA runtime supported
Bus-Id	PCIe address of the GPU
Memory-Usage	Used / Total VRAM
GPU-Util	Current GPU utilization
Pwr:Usage/Cap	Power usage and limit
Perf	Current performance state

2. CHECK DRIVER INITIALIZATION

```
# Check if NVIDIA kernel modules are loaded
lsmod | grep nvidia
nvidia          508559616  56
nvidia_uvm      39649280   4
nvidia_drm      122880    6
nvidia_modeset  1794048    8

# Check driver version
cat /proc/driver/nvidia/version

# Check loaded modules
sudo modinfo nvidia | grep version

# Check NVIDIA persistence mode
nvidia-smi -q | grep "Persistence Mode"
```

EXPECTED OUTPUT

- NVIDIA modules listed in lsmod
- Driver version matches nvidia-smi
- No module loading errors
- Persistence mode should be On for production

COMMON ISSUES

- Module not loaded
- Driver version mismatch
- Failed to initialize NVIDIA driver
- NVRM: GPU not detected

3. CHECK PCIe VISIBILITY

```
# List GPUs via lspci
lspci | grep -i nvidia
17:00.0 3D controller: NVIDIA Corporation
A100-SXM4-40GB (rev a1)

# Detailed PCIe information
lspci -s 17:00.0 -vv

# Check PCIe link status
nvidia-smi -q | grep -A 5 "PCI"
```

WHAT TO CHECK

- GPU is visible in lspci
- Correct PCIe generation (Gen3/4/5)
- PCIe link width (x16 expected)
- No PCIe errors in kernel logs
- GPU not behind unsupported devices

4. CHECK GPU DEVICE FILES

```
$ ls -l /dev/nvidia*
crw-rw-rw- 1 root root 195, 0 /dev/nvidia0
crw-rw-rw- 1 root root 195, 255 /dev/nvidia1
crw-rw-rw- 1 root root 511, 0 /dev/nvidia-uvm
crw-rw-rw- 1 root root 506, 0 /dev/nvidia-modeset
```

KEY DEVICE FILES

• /dev/nvidia0..N	GPU device files
• /dev/nvidia1	Control interface
• /dev/nvidia-uvm	Unified memory
• /dev/nvidia-modeset	Modesetting interface

5. CHECK KERNEL LOGS FOR ERRORS

```
# View recent NVIDIA kernel logs
dmesg | grep -i nvidia

# Follow logs in real time
sudo dmesg -w | grep --line-buffered -i nvidia

# Check Xid errors
dmesg | grep -i "NVRM: Xid"
```

COMMON LOG MESSAGES

- NVRM: Xid 43 - GPU stopped processing
- NVRM: Xid 79 - GPU has fallen off the bus
- NVRM: Xid 31 - GPU memory page fault
- NVRM: GPU at PCI:0000:17:00: GPU is lost
- nvidia: module verification failed

6. COMMON XID ERRORS

Xid	Meaning	Common Cause	Action
13	Graphics engine error	Hardware fault	Check hardware, logs
31	Memory page fault	Invalid memory access	Check application, drivers
43	GPU stopped processing	Long running kernel / hang	Reset GPU, check workload
48	Double bit ECC error	Hardware memory error	Check hardware, replace GPU
79	GPU has fallen off the bus	PCIe or power issue	Check PCIe, power, server
119	Timeout	Long kernel execution	Optimize workload

7. ISOLATE AND TEST GPU

```
# Run basic GPU test
nvidia-smi
nvidia-smi dmon -s u

# Check CUDA visibility
python3 -c "import torch;
print(torch.cuda.is_available())"

# Run NVIDIA diagnostic tools (if installed)
sudo nvidia-bug-report.sh

# Stress test GPU (optional)
sudo gpu-burn -d 0 -tc 60 -b 1024
```

8. TROUBLESHOOTING FLOW



9. REAL-WORLD SCENARIOS

Scenario	Symptoms	Likely Cause	Fix
nvidia-smi fails	Command not found	Driver not installed	Install correct driver
No GPUs in lspci	GPU not visible	PCIe/power issue	Check hardware, BIOS
Xid 43 errors	GPU stops responding	Long kernel / hardware fault	Reset GPU, update driver
Xid 79 errors	GPU not detected	PCIe link failure	Check slot, power, cables
Out of memory	CUDA OOM error	High memory usage	Reduce batch size, use MIG
Driver mismatch	CUDA not available	Incompatible driver/CUDA	Install supported version
Intermittent failures	Random GPU crashes	Power/thermal issue	Check cooling, power limits



@veriqta



@veriqta



@veriqta



@veriqta



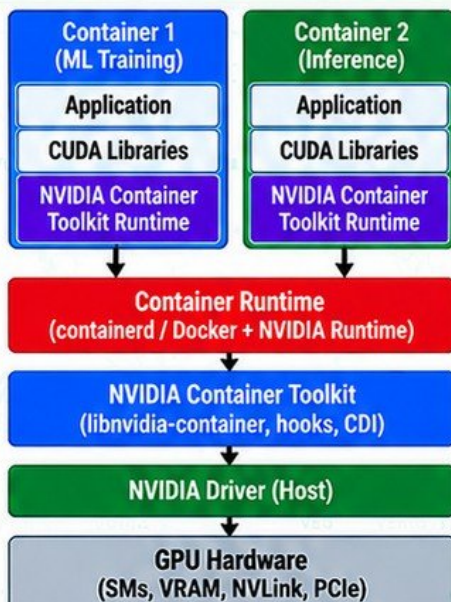
@veriqta



@veriqta

NVIDIA Container Toolkit, CDI, device exposure, runtime configuration, and container security.

1. GPU CONTAINER ARCHITECTURE



Key Functions

- Injects GPU drivers and libraries into containers
- Exposes GPU devices securely
- Uses CDI or legacy runtime configuration
- Isolates workloads at container level
- Works with Docker and containerd

2. NVIDIA CONTAINER TOOLKIT COMPONENTS

Component	Purpose	Key Files / Binaries
libnvidia-container	Prestart hooks and GPU device injection	/usr/bin/nvidia-container-cli
nvidia-container-runtime	OCI runtime for GPU containers	/usr/bin/nvidia-container-runtime
nvidia-ctk	Configuration tool	nvidia-ctk
nvidia-container-toolkit	Main toolkit package	/etc/nvidia-container-runtime/config.toml
CDI (Container Device Interface)	Standard way to expose devices to containers	/etc/cdi/nvidia.yaml
CUDA libraries	User space libraries inside container	/usr/local/cuda, /usr/lib/x86_64-linux-gnu/libcuda.so

3. INSTALL NVIDIA CONTAINER TOOLKIT (UBUNTU)

```
# Add NVIDIA package repository
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey |
sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit.gpg
echo "deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit.gpg]
https://nvidia.github.io/libnvidia-container/stable/deb/$(. /etc/os-release;
echo $ID$VERSION_ID) /" | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt update
sudo apt install -y nvidia-container-toolkit
```

4. GPU CONTAINER RUNTIME CONFIGURATION

```
# Configure containerd with NVIDIA runtime
sudo nvidia-ctk runtime configure --runtime=containerd
sudo systemctl restart containerd

# Verify runtime configuration
sudo nvidia-ctk runtime list
```

Expected Output

NAME	PATH	RUNTIME
nvidia	/usr/bin/nvidia-container-runtime	oci

5. RUN A GPU CONTAINER (DOCKER)

```
# Test GPU access with CUDA container
docker run --rm --gpus all \
nvidia/cuda:12.4.1-base-ubuntu22.04 \
nvidia-smi

# Run with specific GPU
docker run --rm --gpus "device=0" \
nvidia/cuda:12.4.1-base-ubuntu22.04 \
nvidia-smi
```

6. RUN A GPU CONTAINER (CONTAINERD / K8s)

```
# containerd (nerdctl)
nerdctl run --rm --gpus all \
nvidia/cuda:12.4.1-base-ubuntu22.04 \
nvidia-smi

# Kubernetes Pod example
apiVersion: v1
kind: Pod
metadata:
  name: gpu-test
spec:
  containers:
  - name: cuda-container
    image: nvidia/cuda:12.4.1-base-ubuntu22.04
    resources:
      limits:
        nvidia.com/gpu: 1
```

7. CONTAINER DEVICE INTERFACE (CDI)

- CDI provides a standard way to expose GPU devices to containers.
- Allows device injection without using --gpus flag.
- Used by containerd, Kubernetes and other OCI runtimes.
- Improves compatibility and device isolation.

```
# Generate CDI specification
sudo nvidia-ctk cdi generate --output=/etc/cdi/nvidia.yaml
# View CDI devices
sudo cat /etc/cdi/nvidia.yaml
```

CDI EXAMPLE (nvidia.yaml)

```
cdiVersion: 0.5.0
kind: Device
devices:
- name: "gpu0"
  containerEdits:
    deviceNodes:
      - path: /dev/nvidia0
    env:
      - name: NVIDIA_VISIBLE_DEVICES
        value: "0"
```

8. GPU DEVICE EXPOSURE OPTIONS

Method	Description	Example
All GPUs	Expose all GPUs	--gpus all
Specific GPU	Expose selected GPU	--gpus "device=0"
Capabilities	Limit capabilities	--gpus "capabilities=compute,utility"
MIG device	Expose a MIG instance	--gpus "device=MIG-GPU-xxx"
CDI (containerd/K8s)	Use CDI specification	nvidia.com/gpu: 1
Environment variable	Control visible devices	NVIDIA_VISIBLE_DEVICES=0,1

9. CONTAINER SECURITY CONSIDERATIONS

- Use official NVIDIA and trusted base images.
- Avoid running containers as privileged.
- Limit device exposure to only required GPUs.
- Use read-only root filesystem where possible.
- Implement resource limits (GPU, CPU, memory).
- Use Kubernetes namespaces, RBAC and policies.
- Scan images for vulnerabilities.

10. COMMON ISSUES AND FIXES

Issue	Symptom	Cause	Fix
GPU not visible	nvidia-smi fails	Runtime not configured	Install and configure NVIDIA toolkit
Permission denied	Cannot access /dev/nvidia0	Incorrect device permissions	Use NVIDIA runtime or CDI
No CUDA libraries	Application error	Missing libraries in container	Use CUDA base image or mount libraries
Wrong driver version	CUDA error	Host/container mismatch	Use compatible CUDA and driver versions

11. VERIFICATION CHECKLIST

- nvidia-smi works inside container
- Correct GPUs are visible
- CUDA applications run successfully
- Container runtime shows NVIDIA
- CDI devices are available (if using CDI)
- No unnecessary device access
- Security policies are enforced



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

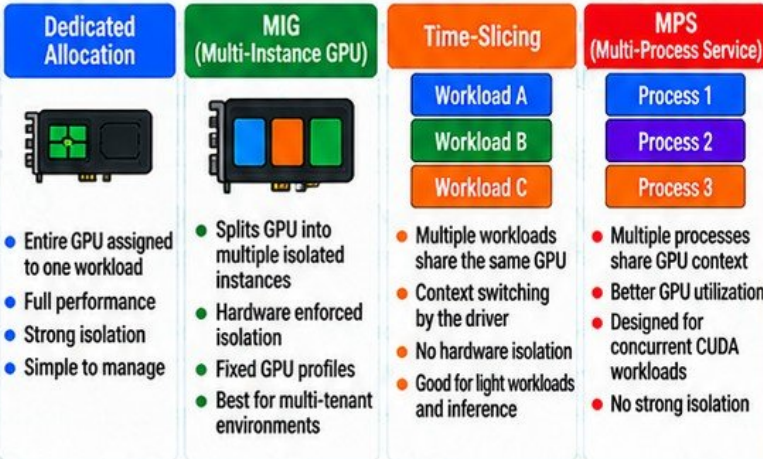


@veriqta

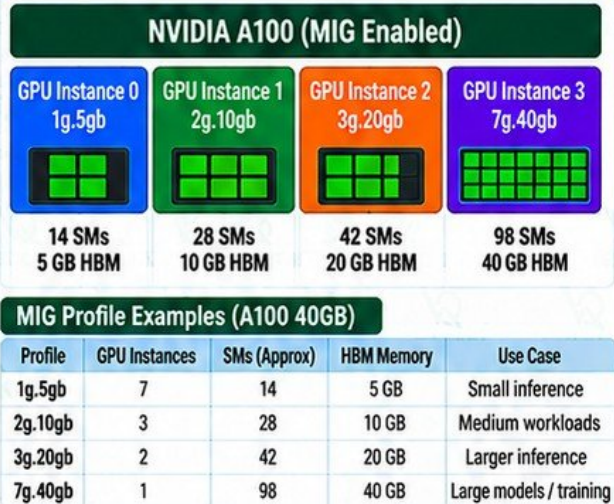
07

MIG, time-slicing, MPS, dedicated allocation, performance isolation, and tenancy trade-offs.

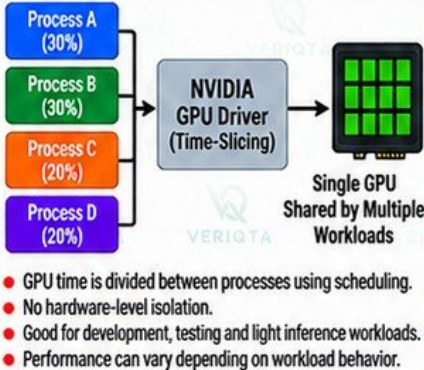
1. GPU SHARING METHODS OVERVIEW



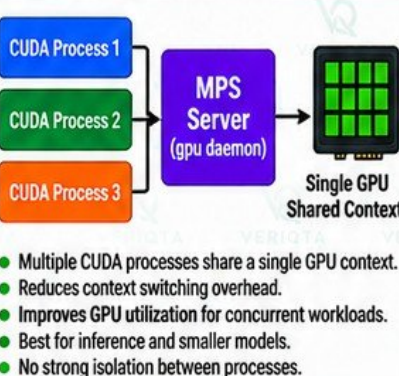
2. MIG ARCHITECTURE AND PROFILES



3. TIME-SLICING ARCHITECTURE



4. MPS ARCHITECTURE



5. COMPARISON OF SHARING METHODS

Method	Isolation Level	Performance	Best Use Case	Trade-offs
Dedicated	Highest	Full Performance	Training, critical workloads	Low utilization
MIG	High (hardware)	Consistent	Multi-tenant production	Fixed profiles, not flexible
Time-Slicing	Low	Variable	Dev, testing, light inference	No isolation, performance interference
MPS	Low to Medium	Better utilization	Concurrent inference	No strong isolation

6. CONFIGURATION EXAMPLES

Enable and configure MIG

```
# Enable MIG mode
sudo nvidia-smi -i 0 -mig 1

# Create MIG instances
sudo nvidia-smi mig -c 19,19,19 -C

# List MIG devices
nvidia-smi -L
```

Start MPS server

```
# Start MPS control daemon
export CUDA_MPS_PIPE_DIRECTORY=/tmp/nvidia-mps
export CUDA_MPS_LOG_DIRECTORY=/tmp/nvidia-mps
nvidia-cuda-mps-control -d

# Check MPS status
echo get_server_list | nvidia-cuda-mps-control
```

Configure GPU time-slicing

```
# Create time-slicing configuration
cat > timeslice.conf << EOF
version: 1
flags:
  - name: "timeslice"
    - value: 1
EOF
sudo nvidia-smi -i 0 -c timeslice.conf
```

Run a container with GPU access

```
# All GPUs
docker run --gpus all \
  nvidia/cuda:12.4.1-base-ubuntu22.04 \
  nvidia-smi

# Specific MIG device (using CDI)
docker run --device=nvidia.com/gpu=1g.5gb \
  nvidia/cuda:12.4.1-base-ubuntu22.04 \
  nvidia-smi
```

7. REAL WORLD SCENARIOS

Scenario	Recommended Method	Reason
Multi-tenant AI platform	MIG	Strong isolation, predictable performance
Shared inference API server	Time-Slicing or MPS	Higher GPU utilization, cost efficient
Dedicated model training	Dedicated GPU	Maximum performance, no interference
Development environment	Time-Slicing	Allow multiple users to share GPU for testing
Mixed workloads (inference + training)	MIG + Dedicated	Isolate critical training, share inference with MIG

8. PERFORMANCE ISOLATION CONSIDERATIONS

- MIG provides the strongest performance isolation using hardware partitioning.
- Time-slicing and MPS can lead to interference between workloads.
- Memory bandwidth, L2 cache and SMs are shared in time-slicing and MPS.
- Monitor GPU utilization, memory usage and latency per workload.
- Choose the right isolation method based on workload type, SLAs and cost.

9. TENANCY TRADE-OFFS

- Higher isolation usually means lower GPU utilization.
- Time-slicing and MPS increase utilization but reduce isolation.
- MIG offers a good balance for multi-tenant environments.
- Dedicated GPUs provide the best performance but higher cost.
- Consider workload characteristics, security requirements and cost when selecting a tenancy model.

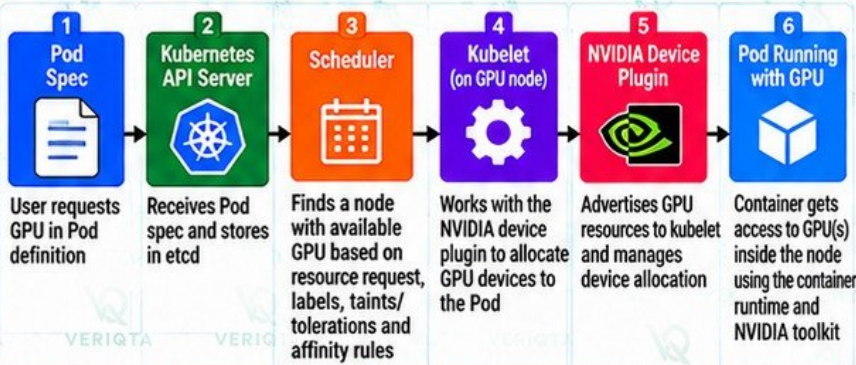
10. BEST PRACTICES

- Use MIG for production multi-tenant workloads.
- Use MPS for concurrent inference workloads.
- Use time-slicing for development and testing.
- Set proper resource limits and monitoring.
- Combine with containers and CDI for easier management.
- Regularly monitor performance and adjust configuration based on workload needs.

09 KUBERNETES GPU CONTROL FLOW

Device plugins, kubelet, extended resources, scheduler decisions, and GPU allocation.

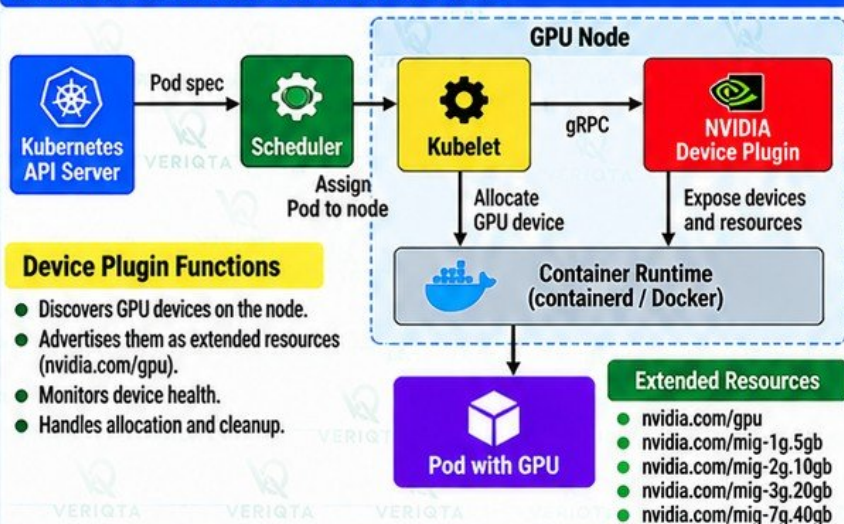
1. HIGH LEVEL CONTROL FLOW



2. KEY COMPONENTS AND ROLES

Component	Role
Kubernetes API Server	Receives and stores Pod specifications with GPU resource requests.
Scheduler	Selects the best node based on available GPU resources, labels, taints and scheduling rules.
Kubelet	Runs on each node, works with the device plugin to allocate GPUs to containers.
NVIDIA Device Plugin	Advertises GPU resources (nvidia.com/gpu) to kubelet and handles device allocation.
Container Runtime (containerd / Docker)	Uses NVIDIA Container Toolkit to make GPU devices available inside containers.
NVIDIA Container Toolkit	Configures the runtime to expose GPU devices and libraries to containers.
GPU Node	Physical or virtual server with NVIDIA GPU(s) and drivers installed.

3. NVIDIA DEVICE PLUGIN ARCHITECTURE



4. POD SPEC EXAMPLE (GPU REQUEST)

```

apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  containers:
    - name: cuda-app
      image: nvidia/cuda:12.4.1-base-ubuntu22.04
      resources:
        limits:
          nvidia.com/gpu: 1
      command: ["nvidia-smi"]
      restartPolicy: Never
  
```

5. SCHEDULER DECISIONS FOR GPU PODS

Factor	Description
Available GPU count	Node must have sufficient free GPUs.
GPU type / model	Use node labels (e.g. nvidia.com/gpu.product).
MIG profiles	Match requested MIG profile (e.g. 1g.5gb).
Taints and tolerations	GPU nodes are usually tainted (e.g. gpu=true:NoSchedule).
Node affinity	Ensure Pod runs on correct GPU nodes.
Resource requests/limits	Scheduler checks nvidia.com/gpu resources.
Topology and NUMA	May influence scheduling for high performance workloads.

6. VERIFY GPU RESOURCES IN KUBERNETES

```

# Check GPU nodes
kubectl get nodes -o wide | grep -i gpu

# Check advertised GPU resources
kubectl describe node <node-name> | grep -A5 "Allocatable"

# List GPU device plugin pod
kubectl get pods -n kube-system | grep nvidia

# Check GPU on a running pod
kubectl exec -it <pod-name> -- nvidia-smi
  
```

7. NODE RESOURCE OUTPUT

```

$ kubectl describe node gpu-node |
  grep -A5 Allocatable
Allocatable:
  cpu:          64
  memory:       251Gi
  nvidia.com/gpu: 4
  nvidia.com/mig-1g.5gb: 7
  nvidia.com/mig-2g.10gb: 3
  
```

8. GPU ALLOCATION EXAMPLE



9. COMMON ISSUES AND FIXES

Issue	Likely Cause	Fix
Pod pending (no GPU)	No available GPUs or wrong request	Check node capacity, labels, taints, resource request.
Node not showing GPU resources	Device plugin not running	Check nvidia-device-plugin pod and logs.
Container cannot see GPU	Runtime not configured	Verify NVIDIA Container Toolkit and runtime class.
MIG profile not available	MIG not enabled on node	Enable MIG and restart device plugin.
Poor performance	Resource contention	Use MIG, dedicated GPUs or proper limits.

10. BEST PRACTICES

- Use node labels to identify GPU type and MIG profiles.
- Use taints and tolerations to control GPU node access.
- Request the right GPU resource (full GPU or specific MIG profile).
- Use resource limits to avoid over-commitment.
- Monitor GPU usage with nvidia-smi, DCGM or Prometheus.
- Use dedicated GPUs for critical production workloads.

11. REAL WORLD USE CASES

- AI/ML model training with dedicated GPUs.
- Multi-tenant inference using MIG.
- Batch inference with time-slicing.
- GPU accelerated data processing pipelines.
- Development and testing with shared GPUs.
- Production workloads with strict isolation and monitoring.

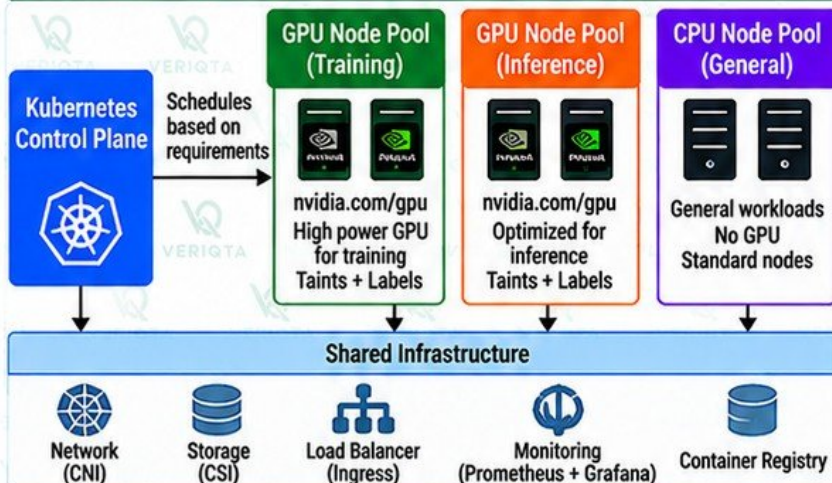
12. KEY TAKEAWAYS

- Kubernetes uses extended resources (nvidia.com/gpu).
- The NVIDIA device plugin exposes GPUs to kubelet.
- Scheduler makes allocation decisions based on available resources.
- Proper configuration ensures containers can access GPUs.
- MIG, time-slicing and resource limits help with isolation and sharing.
- Monitoring and correct scheduling improve reliability and performance.

10 DESIGNING PRODUCTION GPU NODE POOLS

GPU node groups, labels, taints, tolerations, affinity, topology, and workload isolation.

1. PRODUCTION GPU NODE POOL ARCHITECTURE



2. GPU NODE GROUP CONFIGURATION

Component	Example
Node Group Name	gpu-training / gpu-inference
Instance Type	p4d.24xlarge, g5.12xlarge (AWS) A100/H100 (on-prem)
Labels	node-role: gpu-training gpu-type: a100 workload: training
Taints	nvidia.com/gpu=present:NoSchedule
Tolerations	- key: "nvidia.com/gpu" operator: "Equal" value: "present" effect: "NoSchedule"
Node Affinity	RequiredDuringSchedulingIgnoredDuringExecution
Topology	topology.kubernetes.io/zone topology.kubernetes.io/hostname
Auto Scaling	Cluster Autoscaler / Karpenter

3. NODE LABELS AND TAINTS EXAMPLES

Add labels to GPU nodes

```
kubectl label nodes gpu-node-1 \
node-role=gpu-training \
gpu-type=a100 \
workload=training

kubectl label nodes gpu-node-2 \
node-role=gpu-inference \
gpu-type=a100 \
workload=inference
```

Add taints to GPU nodes

```
kubectl taint nodes gpu-node-1 \
nvidia.com/gpu=present:NoSchedule

kubectl taint nodes gpu-node-2 \
nvidia.com/gpu=present:NoSchedule

# Remove taint (if needed)
kubectl taint nodes gpu-node-1 \
nvidia.com/gpu=present:NoSchedule-
```

4. POD TOLERATIONS EXAMPLE

```
tolerations:
- key: "nvidia.com/gpu"
operator: "Equal"
value: "present"
effect: "NoSchedule"
```

5. POD NODE AFFINITY EXAMPLE

```
affinity:
nodeAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
nodeSelectorTerms:
- matchExpressions:
- key: "node-role"
operator: "In"
values: ["gpu-training"]
```

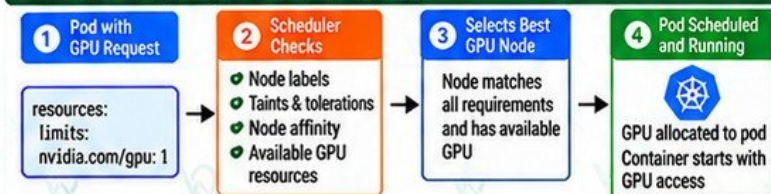
6. GPU WORKLOAD ISOLATION STRATEGIES

Strategy	Purpose	Use Case	Example
Node Labels	Identify GPU nodes	Training or inference	node-role=gpu-training
Taints & Tolerations	Prevent non-GPU workloads	Keep GPU nodes dedicated	nvidia.com/gpu=present:NoSchedule
Node Affinity	Ensure pods run on correct nodes	Training workloads	node-role=gpu-training
Pod Anti-Affinity	Avoid noisy neighbours	High performance training	Avoid scheduling similar workloads on same node
Topology Aware Scheduling	Spread across zones or hosts	High availability	topology.kubernetes.io/zone
Dedicated Node Pools	Full isolation	Sensitive or high-value workloads	Separate node group with GPU instances

7. GPU NODE POOL EXAMPLE WITH KARPENTER (AWS)

```
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
name: gpu-training
spec:
template:
metadata:
node-role: gpu-training
gpu-type: a100
spec:
taints: "nvidia.com/gpu" operator: "In"
values: ["p4d.24xlarge"]
```

8. SCHEDULING FLOW FOR A GPU POD



9. REAL WORLD SCENARIOS

Scenario	Configuration	Benefit
AI model training	Dedicated GPU node pool with taints	Full isolation and performance
LLM inference	Separate inference node pool with different GPU type	Optimized cost and utilization
Multi-tenant platform	Labels + affinity + quotas	Controlled sharing and isolation
Batch workloads	Karpenter auto-scaling GPU nodes	Scale only when needed
Mixed CPU and GPU	Node pools with clear labels	Efficient scheduling and cost control

10. BEST PRACTICES

- Use dedicated GPU node pools for different workloads (training, inference).
- Apply labels, taints and tolerations consistently.
- Use node affinity to ensure correct scheduling.
- Keep GPU nodes free from non-GPU workloads.
- Use topology-aware scheduling for high availability.
- Monitor GPU utilization and right-size your node pools.

11. COMMON ISSUES AND FIXES

Issue	Likely Cause	Fix
Pod pending	No GPU nodes available	Check node pool and auto-scaling
Pod scheduled on wrong node	Missing taints or affinity	Add taints and node affinity
GPU not allocated	Incorrect resource request	Use nvidia.com/gpu in limits
Insufficient GPU capacity	Node pool too small	Increase node pool size or enable auto-scaling
GPU node underutilized	No proper scheduling rules	Use labels, affinity and pod anti-affinity



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

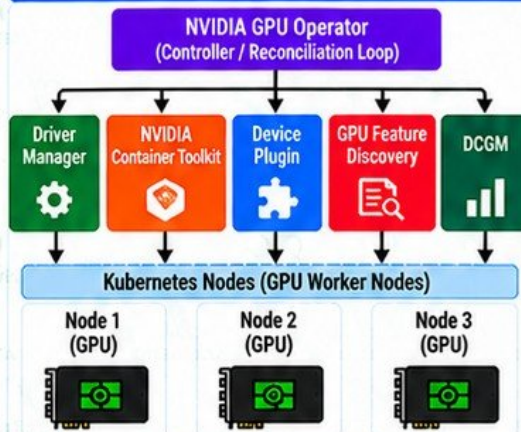
11 NVIDIA GPU OPERATOR ARCHITECTURE

Driver management, toolkit, device plugin, GPU Feature Discovery, DCGM, and operator reconciliation.

1. NVIDIA GPU OPERATOR OVERVIEW

- The NVIDIA GPU Operator automates the deployment, configuration and management of all GPU software components on Kubernetes.
- It ensures correct driver version, container toolkit, device plugin, monitoring and feature discovery.
- Uses Kubernetes Operators (CRDs and controllers) to install, configure and continuously reconcile the GPU software stack.
- Supports bare metal, cloud (EKS, GKE, AKS) and on-premise clusters.

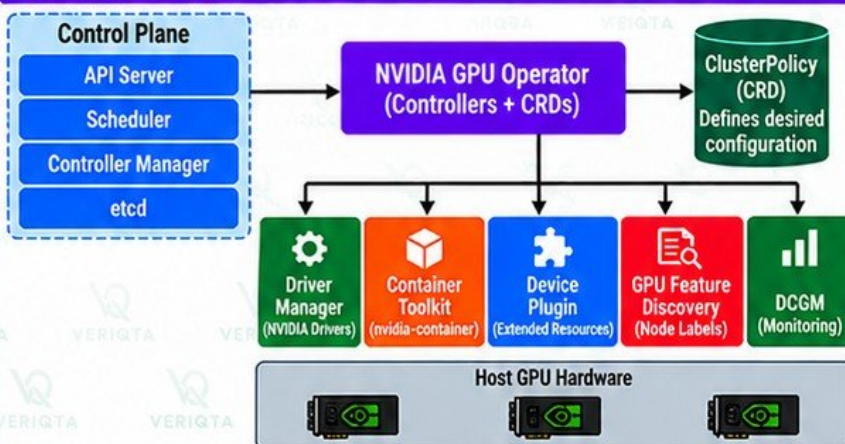
2. HIGH LEVEL ARCHITECTURE



3. COMPONENTS AND FUNCTIONS

Component	Function
NVIDIA GPU Operator	Installs, configures and manages all GPU components using Kubernetes Operators.
Driver Manager	Installs and manages NVIDIA drivers on GPU nodes.
NVIDIA Container Toolkit	Configures container runtime for GPU support.
NVIDIA Device Plugin	Advertises GPU resources to Kubernetes.
GPU Feature Discovery (GFD)	Detects GPU hardware features and adds node labels.
DCGM	Collects GPU metrics, health and telemetry.
Operator Reconciliation	Continuously ensures desired state and handles upgrades and failures.

4. DEPLOYMENT ARCHITECTURE IN A KUBERNETES CLUSTER



5. INSTALL NVIDIA GPU OPERATOR

```
# Add NVIDIA Helm repository
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update

# Create namespace
kubectl create namespace gpu-operator

# Install GPU Operator
helm install gpu-operator nvidia/gpu-operator \
--namespace gpu-operator \
--set driver.enabled=true \
--set toolkit.enabled=true \
--set devicePlugin.enabled=true \
--set gfd.enabled=true \
--set dcm.enabled=true
```

6. CLUSTERPOLICY CONFIGURATION EXAMPLE

```
apiVersion: nvidia.com/v1
kind: ClusterPolicy
metadata:
  name: gpu-operator
  namespace: gpu-operator
spec:
  driver:
    enabled: true
  toolkit:
    enabled: true
  devicePlugin:
    enabled: true
  gfd:
    enabled: true
  dcm:
    enabled: true
  migManager:
    enabled: true
```

7. VERIFY DEPLOYMENT

```
# Check operator pods
kubectl get pods -n gpu-operator

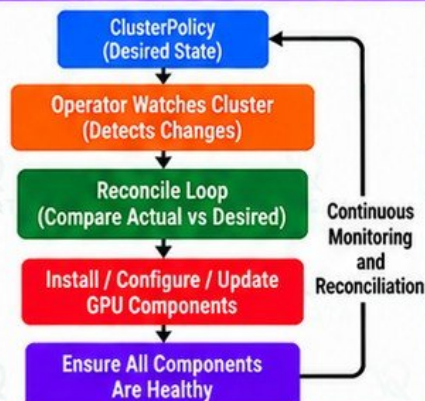
# Check ClusterPolicy status
kubectl get clusterpolicy -n gpu-operator

# Check GPU node labels (from GFD)
kubectl get nodes --show-labels | \
grep nvidia

# Check advertised GPU resources
kubectl describe node <node-name> | \
grep -A 5 "Allocated resources"

# Verify GPU devices inside a container
kubectl run gpu-test --rm -it \
--image=nvidia/cuda:12.4.1-base \
--nvidia.com/gpu=1 -- /bin/bash
nvidia-smi
```

8. OPERATOR RECONCILIATION FLOW



9. BENEFITS

- Automates GPU driver and toolkit management
- Ensures version compatibility
- Simplifies GPU enablement on Kubernetes
- Supports MIG, monitoring and advanced features
- Reduces manual configuration and operational effort
- Handles upgrades, failures and reconciliation
- Works across cloud and on-premise environments

10. COMMON ISSUES AND FIXES

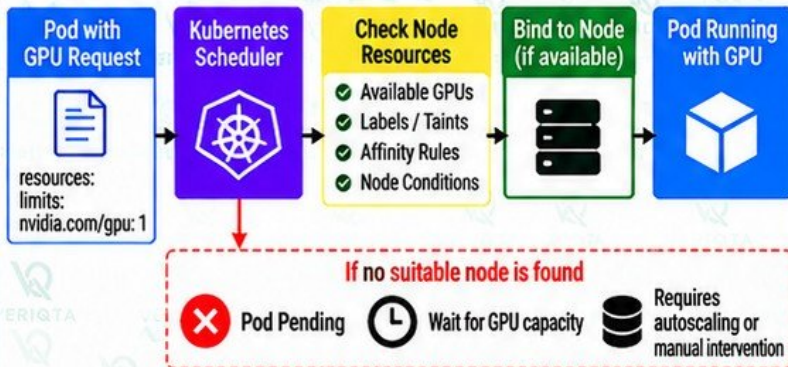
Issue	Cause	Fix
Driver pods failing	Kernel / driver mismatch	Use correct driver version or enable auto-detection
GPU not visible in pods	Device plugin not running	Check device plugin pods and logs
Node labels missing	GFD not running	Enable GFD and check pods
Operator stuck	Image pull or config error	Check operator logs and ClusterPolicy status

11. REAL WORLD USE CASES

- Production Kubernetes clusters with GPUs
- AI/ML model training and inference
- Multi-tenant GPU environments (with MIG)
- Automated GPU lifecycle management
- Cloud GPU clusters (EKS, GKE, AKS)
- On-premise data centers
- Monitoring and observability with DCGM

Pending Pods, unavailable accelerators, GPU fragmentation, autoscaling delays, and capacity planning.

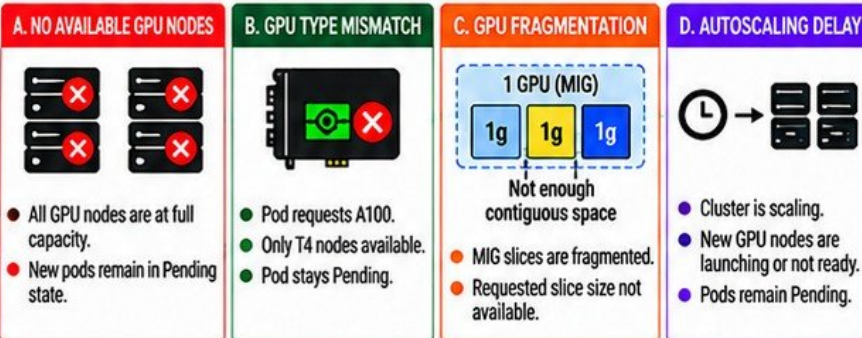
1. GPU POD SCHEDULING FLOW



2. COMMON REASONS FOR PENDING GPU PODS

Reason	Description
No available GPUs	All GPUs are in use or node has no GPUs.
Insufficient GPU type	Requested GPU type (e.g. A100) not available.
Taints not tolerated	GPU nodes have taints and pod lacks tolerations.
Node affinity mismatch	Pod requires specific node labels.
Resource fragmentation	GPUs available but not enough contiguous resources (e.g. MIG slices).
Node not ready	GPU nodes are NotReady or in maintenance.
Autoscaling delay	Cluster is scaling but new GPU nodes are not ready yet.

3. GPU CAPACITY FAILURE SCENARIOS



4. EXAMPLE: PENDING POD OUTPUT

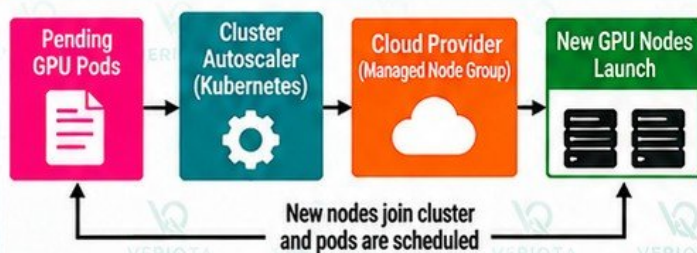
```

$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
gpu-train-1  0/1     Pending   0           5m

$ kubectl describe pod gpu-train-1
...
Events:
  Type     Reason          Age    From          Message
  ----     -
  Warning  FailedScheduling  4m    default-scheduler
           0/5 nodes are available:
           - 3 Insufficient nvidia.com/gpu,
           - 2 node(s) had taint
             {nvidia.com/gpu: NoSchedule}.

```

5. GPU AUTOSCALING ARCHITECTURE



6. KEY FACTORS FOR CAPACITY PLANNING

- Current and projected GPU workload demand.
- GPU type requirements (A100, H100, L4, etc.).
- Use of MIG, time-slicing or dedicated GPUs.
- Training vs inference workload patterns.
- Multi-tenant vs dedicated environments.
- Node provisioning time and cloud limits.
- Regional GPU availability and quotas.
- Cost and utilization targets.

CAPACITY PLANNING FORMULA



Required GPUs =
(Current Demand × Growth Factor)
+ Buffer for Failover

7. CHECK GPU AVAILABILITY AND CAPACITY

```

# Check available GPU nodes
kubectl get nodes -l nvidia.com/gpu.present=true

# Check GPU allocatable resources
kubectl describe nodes <node-name> | grep -A 5 "nvidia.com/gpu"

# Check cluster GPU capacity
kubectl get nodes -o custom-columns=NAME:.metadata.name,\\
GPU:.status.allocatable.nvidia.com/gpu

# Check pending pods
kubectl get pods --field-selector=status.phase=Pending

```

8. FIXING PENDING GPU PODS

- 1 Verify GPU nodes are available and ready.
- 2 Check pod resource requests and GPU type.
- 3 Add correct tolerations and node affinity.
- 4 Scale GPU node group and wait for nodes to be ready.
- 5 Fix resource fragmentation (use MIG or adjust requests).
- 6 Check cloud GPU quotas and limits.
- 7 Monitor autoscaler logs and events.

9. REAL WORLD EXAMPLE SCENARIO

Scenario	Cause	Impact	Solution
AI training job pending	No available A100 GPUs	Training delayed	Scale GPU node group, check quotas
Inference pods pending	Node taints not tolerated	Service unavailable	Add tolerations or fix taints
MIG fragmentation	Slices not contiguous	Pods cannot be scheduled	Adjust MIG profile or recreate MIG layout
Autoscaling slow	Cloud provisioning delay	Pods remain pending	Increase node group size, use multiple AZs

10. BEST PRACTICES

- Use node labels, taints and tolerations for clear GPU scheduling.
- Plan GPU capacity based on real workloads and growth.
- Use MIG or time-slicing to improve utilization.
- Implement cluster autoscaler with proper node group configuration.
- Monitor GPU usage, pending pods and node health.
- Set alerts for GPU capacity and autoscaling delays.
- Regularly review quotas and regional GPU availability.



@veriqta



@veriqta



@veriqta



@veriqta



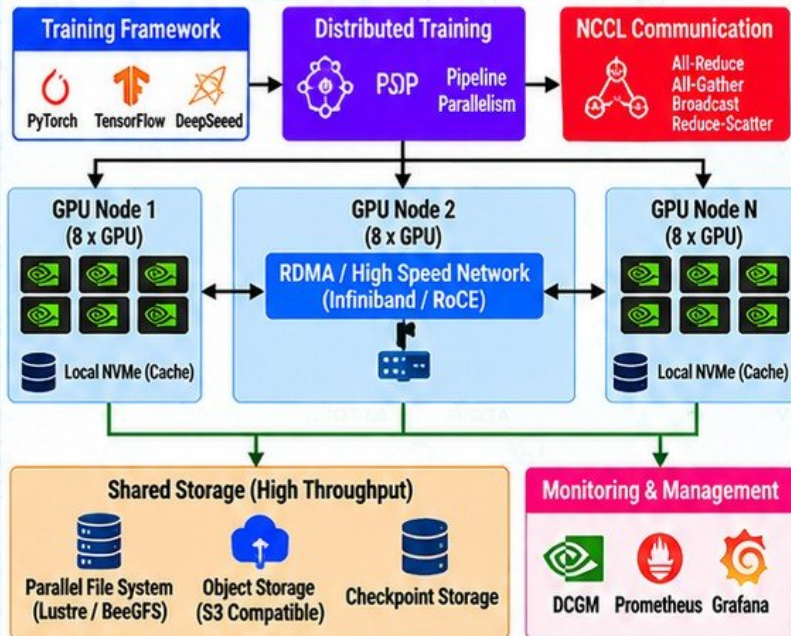
@veriqta



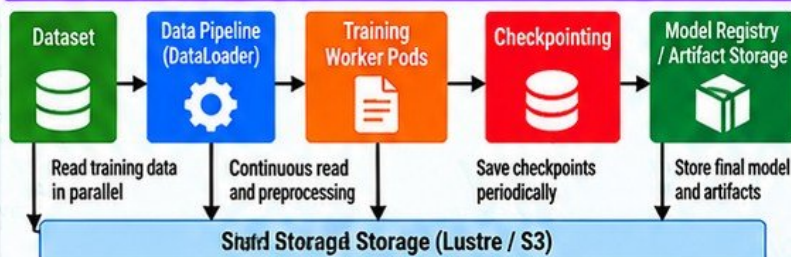
@veriqta

Distributed training, GPU communication, NCCL, RDMA, checkpointing, and storage throughput.

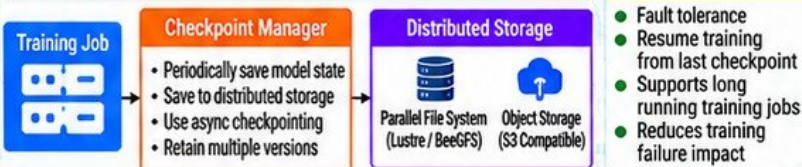
1. HIGH LEVEL ARCHITECTURE



4. TRAINING DATA FLOW



5. CHECKPOINTING ARCHITECTURE



7. EXAMPLE NCCL CONFIGURATION

```
# Environment variables for optimal NCCL performance
export NCCL_DEBUG=INFO
export NCCL_IB_HCA=mlx5
export NCCL_SOCKET_IFNAME=eth0
export NCCL_IB_GID_INDEX=3
export NCCL_NET_GDR_LEVEL=2
export NCCL_ALGO=Ring
export NCCL_PROTO=Simple
export NCCL_NSOKS_PERTHREAD=4
export NCCL_SOCKET_NNTHREAS=2
export NCCL_MIN_NCHANNELS=4
export NCCL_MAX_NCHANNELS=16
```

8. COMMON ISSUES AND FIXES

Issue	Likely Cause	Fix
Slow training performance	NCCL not using RDMA	Check NCCL config, verify IB/RoCE setup
NCCL timeouts	Network issues or firewall	Check network, MTU, firewall rules
GPU out of memory	Model too large or batch size high	Use FSDP/ZeRO, reduce batch size
Checkpoint failures	Storage throughput bottleneck	Use high performance storage (Lustre/BeeGFS)
Nodes not communicating	Incorrect network configuration	Verify IB/RoCE, check NCCL_DEBUG logs

2. DISTRIBUTED TRAINING STRATEGIES

Strategy	Description	Use Case	Pros	Cons
Data Parallel (DDP)	Same model on multiple GPUs, sync gradients (All-Reduce)	Most common training	Simple, scalable	High communication cost
Model Parallel	Split model across GPUs	Very large models	Handles huge models	More complex
Pipeline Parallel	Split layers across GPUs	Very large models	Scales well	Pipeline bubbles
Tensor Parallel	Split tensors across GPUs	Large models (e.g. LLMs)	Faster compute	High communication between GPUs
Fully Sharded Data Parallel (FSDP)	Shards model, grads and optimizer states	Large models with limited GPU memory	Memory efficient	More complex to configure
DeepSpeed ZeRO	Optimizer state partitioning	Very large models	Reduced memory usage	Requires DeepSpeed ecosystem

3. GPU COMMUNICATION TECHNOLOGIES

Technology	Description	Purpose	Benefits	Common Use
NCCL	NVIDIA collective communication library	All-Reduce, All-Gather, etc.	High performance GPU-GPU comms	Multi-GPU training
RDMA (Infiniband)	Direct memory access over network	Low-latency high bandwidth networking	Ultra low latency, high throughput	Large scale multi-node training
RoCE	RDMA over Ethernet	GPU communication	Lower cost alternative	Ethernet based clusters
NVLink / NVSwitch	High speed GPU interconnect	Intra-node GPU traffic	Very high bandwidth	Multi-GPU servers

6. STORAGE THROUGHPUT CONSIDERATIONS

Storage Type	Throughput	Latency	Use Case	Notes
Local NVMe	Very High	Very Low	Dataset cache, temporary files	Use on each GPU node
Lustre	High	Low	Large training datasets	Common in HPC clusters
BeeGFS	High	Low	Large datasets, checkpoints	Scales to thousands of nodes
S3 Compatible	Medium	Medium	Model artifacts, long term storage	Cost effective, high durability

9. BEST PRACTICES

- ✓ Use high speed networking (Infiniband or RoCE) for multi-node training.
- ✓ Optimize NCCL configuration for your cluster.
- ✓ Use distributed storage (Lustre/BeeGFS) for large datasets.
- ✓ Implement regular checkpointing with versioning.
- ✓ Monitor GPU utilization, network bandwidth and storage I/O.
- ✓ Use autoscaling and job queues for efficient resource utilization.
- ✓ Test training at scale before production workloads.
- ✓ Have a disaster recovery plan for long running jobs.

10. REAL WORLD USE CASES

- Training large language models (LLMs) at scale.
- Computer vision model training (autonomous vehicles, medical imaging).
- Recommendation systems with distributed training.
- Multi-modal AI models (text + image + audio).
- Enterprise AI training on Kubernetes clusters.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

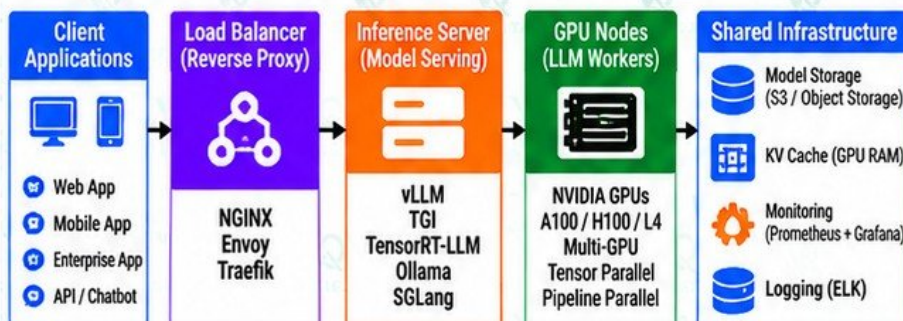


@veriqta

14 LLM INFERENCE INFRASTRUCTURE

Model serving, KV cache, batching, prefill, decoding, concurrency, and latency.

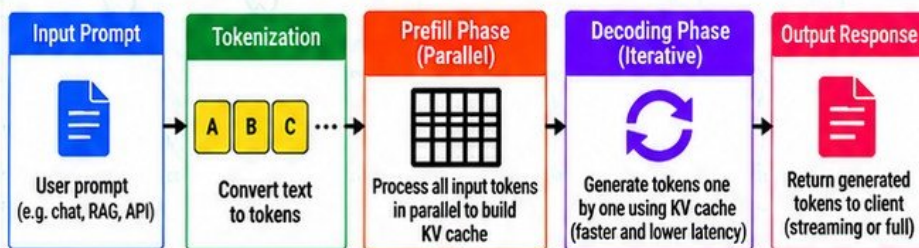
1. LLM INFERENCE ARCHITECTURE



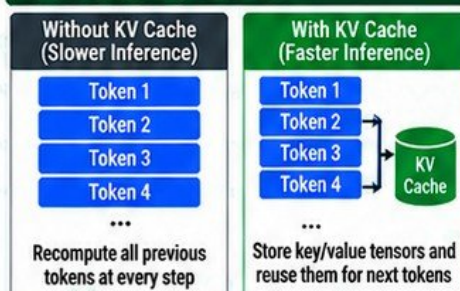
2. KEY COMPONENTS AND ROLES

Component	Role
Model Serving Framework	Loads model, handles requests, scheduling, batching and GPU execution.
GPU	Runs model inference (prefill and decoding) with high throughput.
KV Cache	Stores key/value tensors for fast decoding and reduced latency.
Load Balancer	Distributes requests across multiple inference servers.
Model Storage	Stores model weights and versions (S3, MinIO, Hugging Face, etc.).
Monitoring	Tracks GPU usage, latency, throughput and errors.
Autoscaling	Adds/removes inference pods based on demand and GPU utilization.

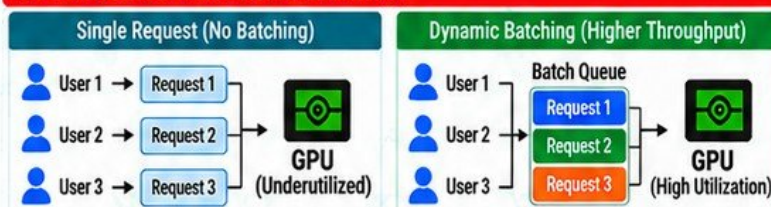
3. INFERENCE WORKFLOW (PREFILL AND DECODING)



4. KV CACHE OVERVIEW



5. BATCHING AND CONCURRENCY



6. LATENCY BREAKDOWN

Stage	Description	Typical Impact
Queueing	Wait in queue before processing	Variable (high under load)
Prefill	Process input tokens in parallel	Higher for long prompts
Decoding	Generate tokens one by one	Largest component
Network	Request/response transfer time	Low to medium
Post-processing	Parsing, formatting, safety checks	Low to medium

7. MODEL SERVING OPTIONS

Framework	Key Features	Use Case
vLLM	High throughput, PagedAttention, dynamic batching	Production LLM serving
TGI (Text Generation Inference)	Easy deployment, Hugging Face integration	General purpose serving
TensorRT-LLM	Optimized for NVIDIA GPUs, high performance	High performance production
Ollama	Simple local serving	Development and small models
SGLang	High-performance, flexible scheduling	Advanced inference workloads

8. EXAMPLE: vLLM DEPLOYMENT (KUBERNETES)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-server
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: vllm
          image: vllm/vllm-openai:latest
          args:
            - --model=/models/llama-3-8b
            - --host=0.0.0.0
            - --port=8000
            - --tensor-parallel-size=2
          resources:
            limits:
              nvidia.com/gpu: 2
```

9. GPU RESOURCE CONSIDERATIONS

- Use high memory GPUs for large models (e.g. H100, A100).
- Plan GPU memory for model weights + KV cache + batching.
- Use quantization (FP8, INT4) when possible.
- Use tensor parallelism for large models.
- Monitor GPU memory usage and OOM errors.
- Use autoscaling for varying demand.
- Use dedicated node pools for inference.

10. COMMON ISSUES AND FIXES

Issue	Likely Cause	Fix
High latency	Small batch size, GPU underutilization	Enable dynamic batching, increase concurrency
Out of memory (OOM)	Large model or KV cache	Use quantization, reduce batch size, use larger GPU
Low GPU utilization	Inefficient batching or too few requests	Tune batch size and concurrency
Slow token generation	Inefficient model config	Use optimized framework (vLLM / TensorRT-LLM)
Model not loading	Insufficient GPU memory	Use smaller model or increase GPU memory
Frequent errors	Invalid input or server overload	Check logs, add rate limiting, scale up

11. BEST PRACTICES

- Use optimized serving frameworks (vLLM/TensorRT-LLM).
- Enable dynamic batching and continuous batching.
- Choose the right GPU type and size for your model.
- Use quantization to reduce memory and increase throughput.
- Monitor latency, throughput, GPU utilization and error rates.
- Implement autoscaling based on real demand.
- Use fast storage (NVMe / S3) for model weights and artifacts.
- Plan capacity for peak usage and multi-tenant workloads.



@veriqta



@veriqta



@veriqta



@veriqta



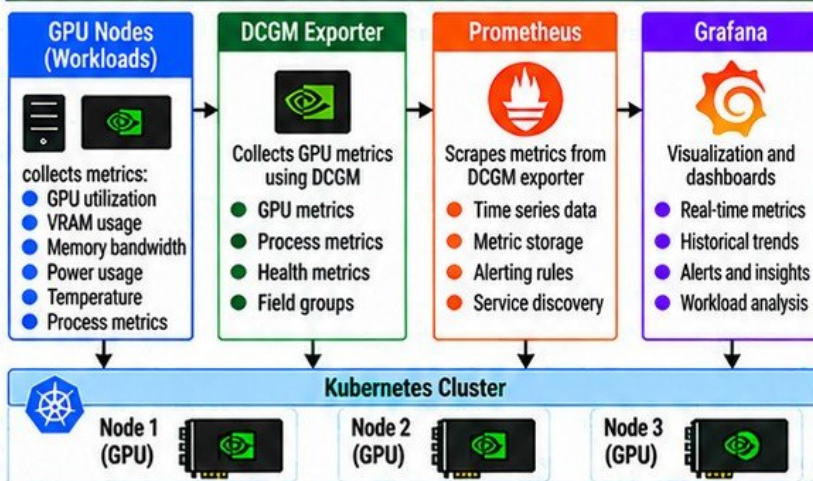
@veriqta



@veriqta

DCGM, Prometheus, Grafana, utilization, VRAM, power, thermals, and workload-level metrics.

1. GPU OBSERVABILITY ARCHITECTURE



2. KEY GPU METRICS AND MEANINGS

Metric Category	Metric / Example	Meaning / Use Case
Utilization	DCGM_FI_DEV_GPU_UTIL	Overall GPU compute utilization (%)
VRAM	DCGM_FI_DEV_FB_USED	Used GPU memory (MiB)
Memory Bandwidth	DCGM_FI_DEV_FB_* (READ/WRITE)_THROUGHPUT	Memory read/write throughput (MB/s)
Power	DCGM_FI_DEV_POWER_USAGE	Current power usage (W)
Temperature	DCGM_FI_DEV_GPU_TEMP	GPU temperature (°C)
Clocks	DCGM_FI_DEV_SM_CLOCK	Graphics/SM clock speed (MHz)
Processes	DCGM_FI_PROF_GR_ENGINE_ACTIVE	Per-process GPU usage
Errors	DCGM_FI_DEV_XID_ERRORS	GPU error count (Xid)
Health	DCGM_FI_DEV_HEALTH	Overall GPU health status

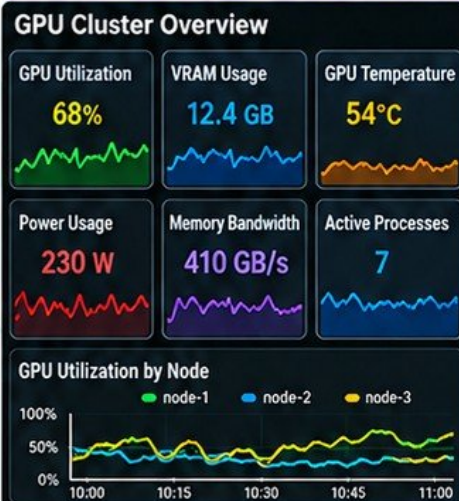
3. DCGM EXPORTER DEPLOYMENT (KUBERNETES)

```
# Deploy DCGM exporter
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: dcm-exporter
  namespace: monitoring
spec:
  selector:
    matchLabels:
      name: dcm-exporter
  template:
    metadata:
      labels:
        name: dcm-exporter
    spec:
      containers:
        - name: dcm-exporter
          image: nvcr.io/nvidia/k8s/dcm-exporter:latest
          ports:
            - containerPort: 9400
          name: metrics
          resources:
            limits:
              nvidia.com/gpu: 0 # no GPU required for exporter
            requests:
              cpu: 100m
              memory: 256Mi
```

4. PROMETHEUS SCRAPE CONFIGURATION

```
scrape_configs:
- job_name: 'dcm-exporter'
  kubernetes_sd_configs:
  - role: endpoints
  relabel_configs:
  - source_labels: [__meta_kubernetes_service_name]
    action: keep
    regex: dcm-exporter
  metric_relabel_configs:
  - source_labels: [__name__]
    regex: "DCGM_.*"
    action: keep
```

5. GRAFANA DASHBOARD EXAMPLE



6. WORKLOAD-LEVEL GPU METRICS

Metric	Description	Use Case
Per-process GPU utilization	GPU usage per container/process	Identify heavy workloads
Per-process VRAM usage	Memory used by each process	Detect memory leaks
SM occupancy	Streaming multiprocessor usage	Optimize performance
Kernel execution time	Time spent in GPU kernels	Find bottlenecks
GPU memory throughput	Read/write throughput per workload	Tune data pipelines
Inference tokens/sec	Model-specific metric	Monitor LLM performance
Training samples/sec	Training performance metric	Track training efficiency

7. ALERTING EXAMPLES

Alert	Condition	Action
High GPU Utilization	GPU > 90% for 10 minutes	Scale or investigate
High VRAM Usage	VRAM > 90%	Check for memory leak or scale
High Temperature	GPU temp > 85°C	Investigate cooling or reduce load
High Power Usage	Power > threshold	Check workload and cooling
GPU Xid Errors	Xid errors > 0	Investigate hardware or driver issues
Exporter Down	No metrics for 5 minutes	Restart DCGM exporter

8. USEFUL QUERIES (PROMQL)

```
# GPU utilization (average per node)
avg(DCGM_FI_DEV_GPU_UTIL) by (instance)

# VRAM usage (used vs total)
DCGM_FI_DEV_FB_USED / DCGM_FI_DEV_FB_TOTAL

# GPU temperature
DCGM_FI_DEV_GPU_TEMP

# Power usage
DCGM_FI_DEV_POWER_USAGE

# Per-process GPU utilization
DCGM_FI_PROF_GR_ENGINE_ACTIVE by (process_name)

# Top 5 GPUs by memory usage
topk(5, DCGM_FI_DEV_FB_USED)
```

9. COMMON GPU ISSUES AND FIXES

Issue	Possible Cause	Fix
No GPU metrics in Prometheus	DCGM exporter not running	Check pod status and service discovery
Metrics show 0 utilization	No active GPU workloads	Verify workloads and pods
VRAM usage constantly high	Memory leak or idle processes	Check running processes and restart
GPU temperature high	Poor cooling or high load	Check airflow, reduce load
DCGM exporter crashlooping	Driver mismatch or permissions	Verify driver and container runtime
Missing process metrics	DCGM permissions not set	Enable required DCGM field groups

10. BEST PRACTICES

- Use DCGM exporter with Prometheus and Grafana for full visibility.
- Monitor utilization, VRAM, power, temperature and errors.
- Set up alerts for high utilization, temperature and Xid errors.
- Track workload-level metrics to identify bottlenecks.
- Use dashboards for real-time and historical analysis.
- Monitor inference and training specific metrics (tokens/sec, samples/sec).
- Keep GPU drivers and DCGM exporter versions compatible.
- Regularly review metrics and optimize resource usage.
- Integrate with logging and tracing for deeper analysis.
- Plan capacity based on real usage metrics.



@veriqta



@veriqta



@veriqta



@veriqta



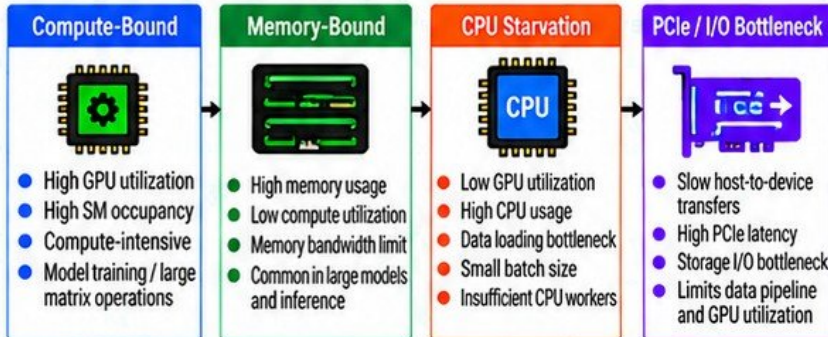
@veriqta



@veriqta

Compute-bound vs memory-bound workloads, CPU starvation, PCIe transfers, I/O, and profiling.

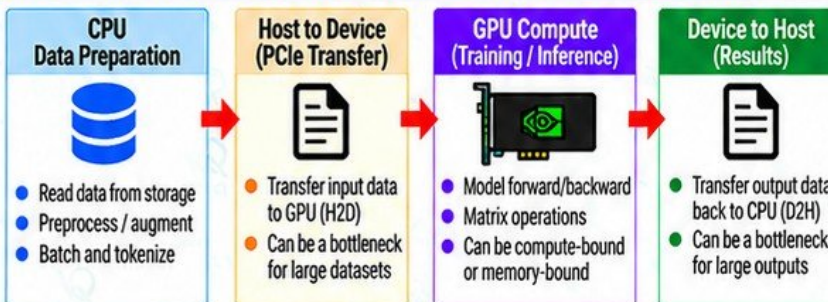
1. TYPES OF GPU BOTTLENECKS



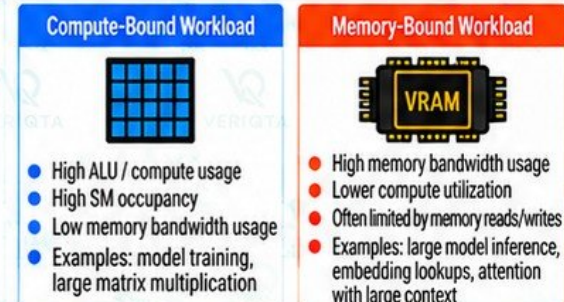
2. BOTTLENECK SYMPTOMS AND INDICATORS

Bottleneck Type	Key Symptoms	Metrics to Check
Compute-Bound	High GPU utilization, low memory usage	GPU utilization, SM occupancy, FLOPS
Memory-Bound	High memory usage, lower compute utilization	VRAM usage, memory bandwidth, cache hit rate
CPU Starvation	Low GPU utilization, high CPU usage	CPU usage, data loader performance, batch size
PCIe Transfer	Low GPU utilization during data transfer	PCIe throughput, h2d/d2h latency
Storage I/O	GPU waiting for data	Disk I/O throughput, read/write latency
Kernel / Driver	Frequent kernel launch, driver issues	kernel time, driver logs, GPU errors

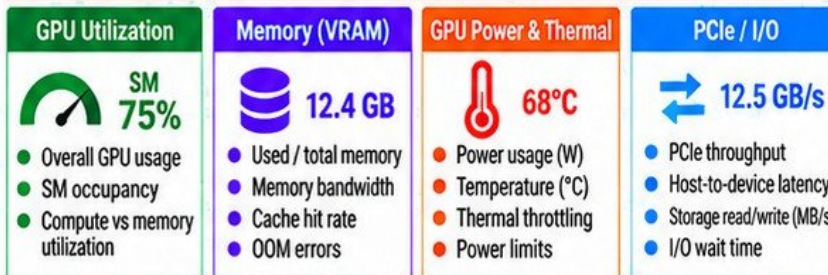
3. WORKLOAD EXECUTION FLOW AND POTENTIAL BOTTLENECKS



4. COMPUTE-BOUND VS MEMORY-BOUND



5. MONITORING KEY METRICS



6. USEFUL PROFILING TOOLS

Tool	Use Case	Key Commands / Notes
nvidia-smi	Real-time GPU metrics	nvidia-smi nvidia-smi dmon
DCGM	Detailed GPU monitoring	dcmi stats DCGM exporter (Prometheus)
Nsight Systems	End-to-end performance analysis	nsys profile python train.py
Nsight Compute	Kernel-level profiling	ncu python train.py
PyTorch Profiler	Training performance analysis	torch.profiler (built-in)
nvprof (legacy)	GPU kernel profiling	nvprof python train.py
iostat / nmon	System I/O monitoring	iostat -x 1 nmon

7. EXAMPLE: IDENTIFYING A BOTTLENECK

```
# Check GPU utilization and memory
nvidia-smi
+-----+-----+-----+-----+-----+
| GPU | Name | Util% | Memory-Usage | Temp | Power |
+-----+-----+-----+-----+-----+
| 0   | A100 | 98%   | 18.2GiB / 40GiB | 72°C | 285W  |
+-----+-----+-----+-----+-----+

# Monitor in real-time
nvidia-smi dmon -s pucmem -d 2

# Profile with Nsight Systems
nsys profile -o gpu_profile python train.py

# Profile with Nsight Compute
ncu --set full python train.py

# Check CPU and I/O
top
iostat -x 1
nmon -s u -s d -s n -c 10
```

9. COMMON ISSUES AND FIXES

Issue	Cause	Fix
Low GPU utilization	CPU data loading slow	Increase num_workers, optimize data pipeline
High memory usage	Large batch size / memory leak	Reduce batch size, optimize model memory usage
Slow training/inference	Compute or memory bound	Profile with Nsight, optimize kernels, use mixed precision
High PCIe transfer time	Large data transfers	Use data prefetching, compression, faster storage
GPU thermal throttling	High temperature	Improve cooling, reduce power limit
OOM errors	Insufficient VRAM	Reduce batch size, use model parallelism, optimize memory
High I/O wait	Slow storage	Use faster storage (NVMe), cache datasets, parallel I/O

8. INTERPRETING RESULTS

Observation	Likely Cause
High GPU util, low memory	Compute-bound workload
Low GPU util, high memory	Memory-bound workload
Low GPU util, high CPU util	CPU starvation / data loading issue
Low GPU util, high PCIe usage	PCIe transfer bottleneck
GPU waiting on I/O	Storage I/O bottleneck
High temperature, lower perf	Thermal throttling
Frequent kernel launches	Inefficient workload / small batch size

10. BEST PRACTICES

- Monitor GPU, CPU, memory, I/O and network metrics together.
- Use profiling tools (Nsight Systems/Compute) for detailed analysis.
- Right-size batch size and number of workers.
- Optimize data pipeline and use prefetching.
- Use mixed precision and efficient kernels.
- Monitor thermal and power to avoid throttling.
- Use high-performance storage (NVMe / parallel file systems).
- Regularly review logs and metrics for early detection.
- Set alerts for low utilization, high temperature, OOM and errors.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

16

GPU PRODUCTION FAILURE SCENARIOS

GPU OOM, Xid errors, unhealthy devices, thermal throttling, failed nodes, and incident mitigation.

1. COMMON GPU PRODUCTION FAILURE SCENARIOS

GPU OOM (Out of Memory)	Xid Errors	Unhealthy Devices	Thermal Throttling	Failed Nodes
 - CUDA OOM errors - Memory fragmentation - Unreleased tensors - Large batch / model - Insufficient VRAM	 - Xid 31 (GPU error) - Xid 43 (timeout) - Xid 79 (fallen off) - Driver or HW issues - Requires reset / recovery	 - Failing health checks - ECC errors - Bad memory sectors - Firmware issues - Device not responding	 - High temperature - Reduced clock speed - Performance drop - Cooling issues - Hotspots	 - Node not ready - GPU not detected - Kubelet issues - Driver mismatch - Hardware failure

2. IMMEDIATE INCIDENT RESPONSE

- 1 Identify and confirm the failure (logs, metrics, alerts).
- 2 Isolate the affected GPU or node.
- 3 Check error messages and system logs.
- 4 Determine the root cause (hardware, driver, config, etc.).
- 5 Apply the right fix or workaround.
- 6 Validate recovery and monitor closely.
- 7 Document the incident and update runbooks.

3. GPU OOM: DIAGNOSIS AND FIX

Common Causes	Key Checks	Fixes
- Model too large for available VRAM - Memory fragmentation - Leaked tensors / not freed - High batch size - Multiple processes on same GPU	- Check GPU memory usage (nvidia-smi) - Review application logs - Look for memory leaks - Verify batch size and model size	✓ Reduce batch size ✓ Enable gradient checkpointing ✓ Optimize model / inference ✓ Free unused tensors ✓ Use memory fractioning

4. XID ERRORS: COMMON CODES

Xid Code	Meaning	Typical Cause	Action
31	GPU memory fault	Memory error / ECC	Check hardware, run memory tests
43	Timeout	Long kernel / hang	Check workload, increase timeout
79	GPU fallen off	PCIe / reset issue	Check connection, restart driver
13	Graphics exception	Driver / software	Update driver, check logs

5. UNHEALTHY DEVICES: DETECTION AND HANDLING

Detection	Common Signs	Handling
 nvidia-smi health check  DCGM diagnostics  Kubernetes device plugin  Monitoring alerts	 ECC errors - Unknown / retired page - Temperature anomalies - Frequent resets	 Drain node - Stop workloads - Run diagnostics - Replace / RMA if needed

6. THERMAL THROTTLING: MONITOR AND PREVENT

Key Metrics	Thresholds (example)	Actions
 GPU temperature Fan speed Clock speed Power usage	 Normal: < 75°C Warning: 75-85°C Critical: > 85°C	✓ Improve cooling ✓ Check airflow ✓ Clean dust ✓ Adjust power limits

7. FAILED NODES: TROUBLESHOOTING FLOW

Node Not Ready	GPU Not Detected	Driver Mismatch	Hardware Failure
 - Check kubelet logs - Verify GPU driver - Check systemd	 - Check device plugin - Verify driver version - Check kernel logs	 - Check CUDA version - Update drivers - Reboot node	 - Run diagnostics - Check PCIe / power - Replace hardware

8. LOGS AND TOOLS FOR INVESTIGATION

Key Commands	Monitoring Tools
<pre>nvidia-smi nvidia-smi dmon -s u nvidia-bug-report.sh kubectl logs <pod> dmesg grep -i nvidia journalctl -u kubelet dcmgi diagnostics</pre>	 DCGM (GPU metrics)  Prometheus (metrics/scraping)  Grafana (visualization)  Kubernetes (node/pod health)

9. PREVENTION BEST PRACTICES

Infrastructure	Operational
✓ Use proper cooling and power planning ✓ Set GPU utilization and temperature alerts ✓ Keep drivers and firmware updated ✓ Run regular health checks and diagnostics ✓ Plan for capacity and redundancy	✓ Use resource limits and quotas ✓ Monitor with Prometheus + Grafana ✓ Enable logging and alerting ✓ Test failure scenarios regularly ✓ Keep runbooks up to date

10. KEY TAKEAWAYS

Monitor Early	Understand Root Cause	Apply Fixes	Prevent Recurring Issues	Keep Running
HEALTHY GPUS = STABLE WORKLOADS + HIGHER PRODUCTIVITY + LOWER COSTS				



@veriqta



@veriqta



@veriqta



@veriqta



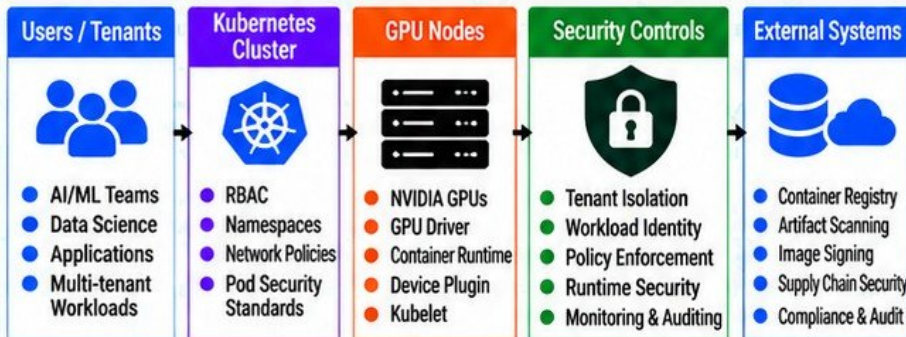
@veriqta



@veriqta

Tenant isolation, privileged components, driver attack surface, workload identity, and supply-chain controls.

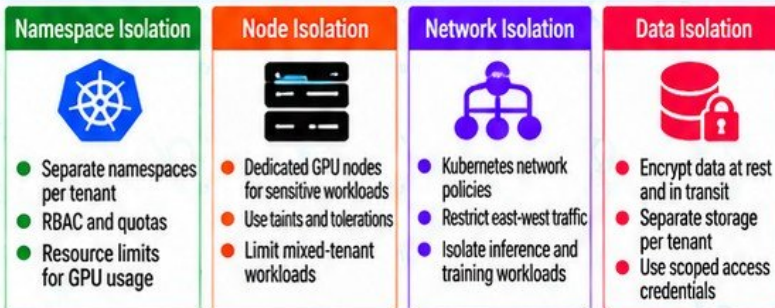
1. GPU SECURITY ARCHITECTURE



2. KEY SECURITY RISKS

Risk	Description
Tenant Isolation	Multiple tenants sharing the same GPU resources can lead to data leakage or side-channel attacks.
Privileged Components	GPU drivers, device plugins and operator components often require elevated privileges.
Driver Attack Surface	NVIDIA drivers, kernel modules and user-space libraries can be exploited if not patched.
Container Escape	Compromised containers may attempt to access GPU devices or host resources.
Supply Chain Risks	Untrusted container images, drivers or CUDA libraries can introduce malware.
Workload Identity	Weak identity and access controls can allow unauthorized GPU usage.
Data Exposure	Model weights, training data and inference inputs may be exposed.

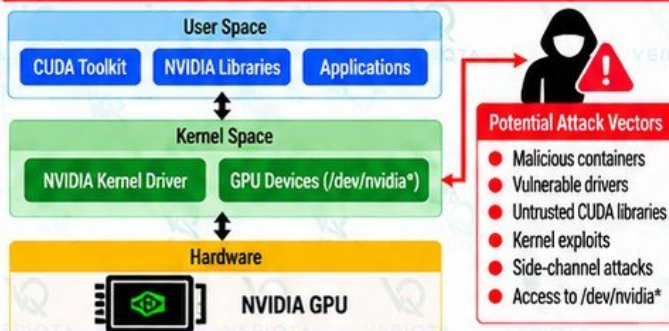
3. TENANT ISOLATION STRATEGIES



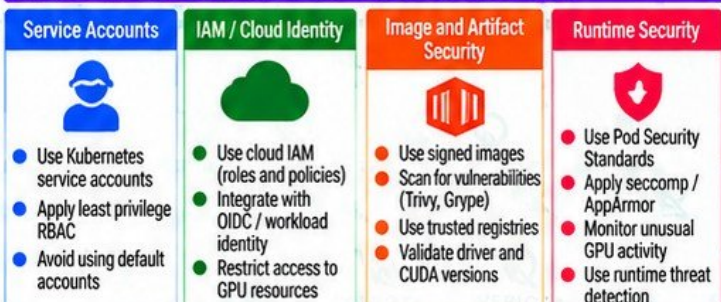
4. PRIVILEGED COMPONENTS AND CONTROLS

Component	Privilege Level	Security Best Practices
NVIDIA Driver	Kernel (High)	Keep updated, use trusted sources, restrict host access
Device Plugin	High	Run with minimal required permissions, audit access
GPU Operator	High	Use latest version, limit RBAC, monitor changes
Kubelet	High	Secure configuration, restrict API access
Container Runtime (containerd)	High	Use runtime security policies (AppArmor/SELinux)
CUDA Libraries	Medium	Use trusted images, scan for vulnerabilities
Monitoring Tools (DCGM/Prometheus)	Medium	Restrict network access, use authentication

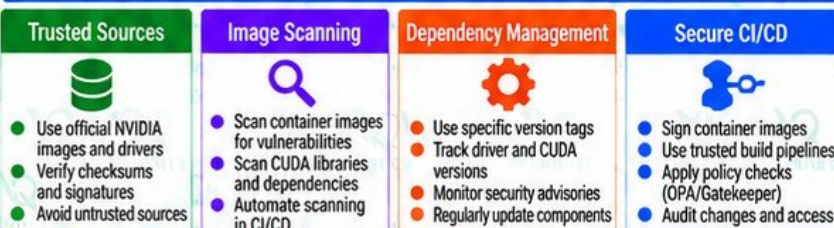
5. DRIVER ATTACK SURFACE



6. WORKLOAD IDENTITY AND ACCESS CONTROL



7. SUPPLY-CHAIN SECURITY FOR GPU INFRASTRUCTURE



8. SECURITY MONITORING AND AUDITING

What to Monitor	Tools and Solutions
- GPU usage per tenant - Access to /dev/nvidia* - Driver and kernel logs - Unusual GPU process activity - Failed access attempts - Configuration changes	- DCGM (GPU metrics) - Prometheus + Grafana - Falco (runtime monitoring) - Audit logs (Kubernetes) - SIEM integration (e.g. ELK) - Alerting for suspicious activity

9. INCIDENT RESPONSE FOR GPU SECURITY

- Detect:** Identify unusual GPU activity or security alerts.
- Contain:** Isolate affected workloads or nodes.
- Investigate:** Check logs, driver status, and access history.
- Mitigate:** Apply patches, revoke access, or restart components.
- Recover:** Restore normal operations and monitor.
- Learn:** Document the incident and improve security controls.

10. BEST PRACTICES

- Use dedicated GPU nodes for sensitive workloads.
- Keep GPU drivers, CUDA and operator components updated.
- Implement strong tenant isolation (namespaces, taints, network policies).
- Use least privilege access and strong workload identity.
- Scan and sign container images and artifacts.
- Monitor GPU usage and access logs.
- Apply runtime security (seccomp, AppArmor, Falco).
- Have an incident response plan for GPU security events.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

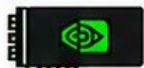
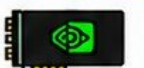
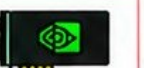


@veriqta

18

Accelerator selection, utilization, reservations, queueing, right-sizing, scaling, and cost per workload.

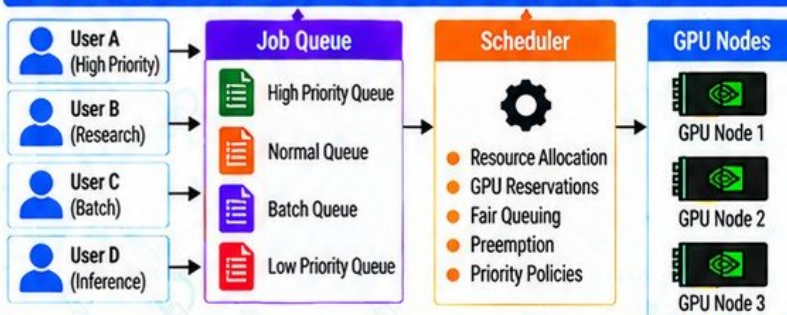
1. ACCELERATOR SELECTION FOR DIFFERENT WORKLOADS

Training (LLMs / Foundation Models)	Fine-tuning (Moderate Scale)	Inference (Real-time / Production)	Batch / Offline (Non-urgent jobs)	General Purpose (Multi-workload)
 • H100 / H200 • High memory (80GB+) • High interconnect (NVLink, NVSwitch) • Best for large-scale training	 • A100 / L40S • Good performance and cost balance • Suitable for fine-tuning and smaller models	 • L4 / L40S / A10G • High inference throughput • Lower latency • Good cost efficiency	 • A10G / L4 • Cost-optimized • Good for batch processing • Can use spot instances	 • H100 / A100 / L40S • Supports training, fine-tuning, inference • Choose based on budget and scale

2. GPU UTILIZATION METRICS

Metric	Description
GPU Utilization	Percentage of time GPU is busy (execute compute).
SM Utilization	Streaming multiprocessor usage (compute activity).
Memory (VRAM) Usage	Amount of GPU memory used.
Memory Bandwidth	Rate of memory read/write (GB/s).
GPU Power Usage	Current power consumption (W).
GPU Temperature	Current GPU temperature (°C).
GPU Duty Cycle	Overall workload efficiency.

3. WORKLOAD QUEUEING AND RESOURCE MANAGEMENT



4. RIGHT-SIZING AND SCALING STRATEGIES

- ✓ Start with actual workload requirements (model size, batch size, latency).
- ✓ Use GPU utilization data to identify over-provisioned or underutilized resources.
- ✓ Right-size instances based on real usage patterns.
- ✓ Use horizontal scaling (more nodes) for high throughput.
- ✓ Use vertical scaling (larger GPUs) for memory-intensive workloads.
- ✓ Implement autoscaling based on queue length, utilization, or custom metrics.
- ✓ Use spot instances for batch and non-critical workloads.
- ✓ Set sensible limits and quotas to avoid resource starvation.

5. GPU CAPACITY PLANNING STEPS

- 1 Understand workloads (training, fine-tuning, inference, batch, multi-tenant).
- 2 Estimate resource requirements (GPU type, memory, CPU, storage, network).
- 3 Analyze current utilization and future growth.
- 4 Plan for reservations and quotas.
- 5 Design queueing and scheduling strategy.
- 6 Choose scaling strategy (on-demand, autoscaling, spot).
- 7 Calculate total cost per workload.
- 8 Plan for high availability and disaster recovery.
- 9 Continuously monitor and optimize.
- 10 Review and adjust based on usage and business needs.

6. EXAMPLE GPU COST COMPARISON (CLOUD)

GPU Type	Memory	Typical Use Case	On-Demand Cost (USD/hr)	Spot Cost (USD/hr)
NVIDIA L4	24 GB	Inference, lightweight workloads	\$0.60	\$0.20
NVIDIA A10G	24 GB	Inference, fine-tuning	\$1.00	\$0.35
NVIDIA L40S	48 GB	Inference, fine-tuning	\$1.50	\$0.50
NVIDIA A100	40/80 GB	Training, large models	\$2.50	\$0.80
NVIDIA H100	80 GB	Large model training	\$4.00	\$1.50

Note: Prices are examples (e.g., AWS) and vary by region and provider.

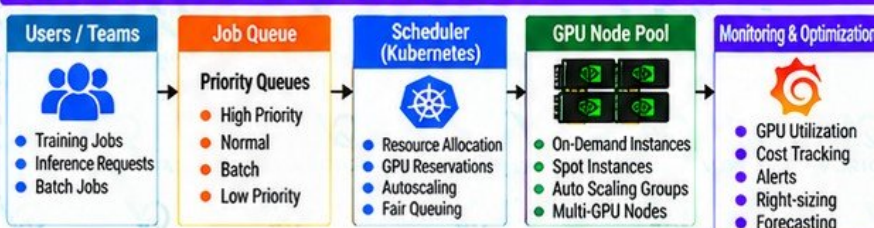
7. COST PER WORKLOAD EXAMPLE

Workload	GPU Type	GPU Hours	Cost per Hour (USD)	Total Cost (USD)
Model Training	H100	1,000	\$4.00	\$4,000
Fine-tuning	A100	500	\$2.50	\$1,250
Inference (Production)	L4	2,000	\$0.60	\$1,200
Batch Processing	A10G	1,000	\$1.00	\$1,000

8. UTILIZATION AND COST OPTIMIZATION TIPS

- ✓ Increase GPU utilization (reduce idle time).
- ✓ Use dynamic batching for inference.
- ✓ Right-size GPU instances to workload needs.
- ✓ Use queueing to avoid GPU starvation.
- ✓ Use spot instances for non-critical workloads.
- ✓ Enable autoscaling based on real metrics.
- ✓ Monitor cost per workload and set budgets.
- ✓ Use mixed instance types where possible.
- ✓ Stop unused resources and clean up.
- ✓ Use model optimization (quantization, smaller models) to reduce GPU requirements.
- ✓ Regularly review usage, costs, and forecasts.

9. CAPACITY PLANNING EXAMPLE ARCHITECTURE



10. KEY TAKEAWAYS

- ✓ Choose the right GPU for the right workload.
- ✓ Monitor and improve GPU utilization.
- ✓ Use reservations, queueing, and quotas to manage demand.
- ✓ Right-size and scale based on real usage.
- ✓ Use spot instances and optimization techniques to reduce cost.
- ✓ Track cost per workload and set budgets.
- ✓ Continuously monitor, forecast, and adjust capacity.
- ✓ Efficient GPU planning improves performance, reduces cost, and ensures fair access for all users.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

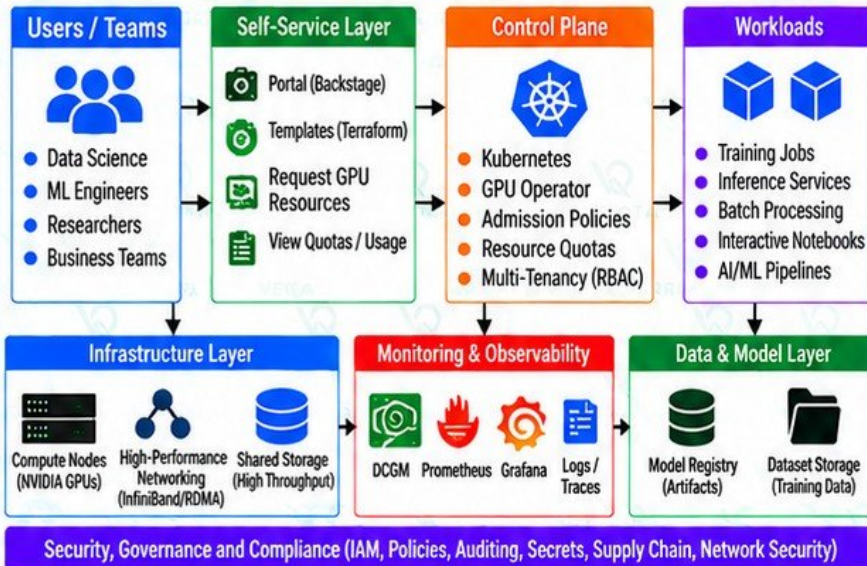


@veriqta

19

Reference architecture, workload onboarding, self-service, operational ownership, observability, SLOs, and disaster recovery.

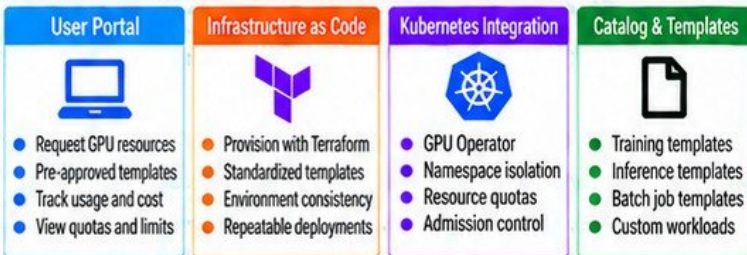
1. ENTERPRISE GPU PLATFORM REFERENCE ARCHITECTURE



2. WORKLOAD ONBOARDING FLOW



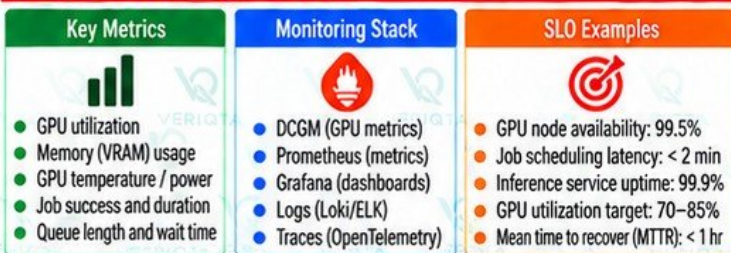
3. SELF-SERVICE PLATFORM COMPONENTS



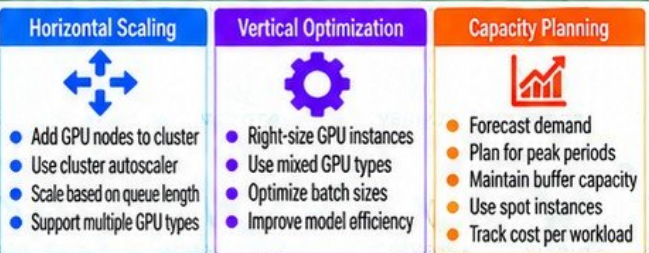
4. OPERATIONAL OWNERSHIP MODEL



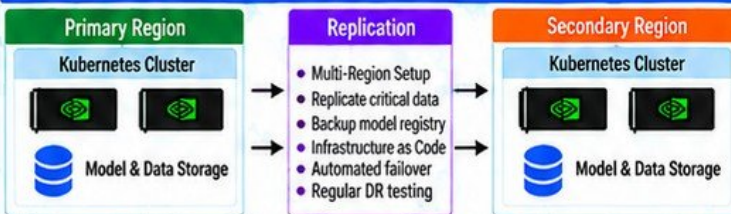
5. OBSERVABILITY AND SLOs



6. SCALING STRATEGIES



7. DISASTER RECOVERY ARCHITECTURE



8. COST MANAGEMENT



9. SECURITY AND GOVERNANCE (HIGH LEVEL)



10. KEY TAKEAWAYS

- Use a reference architecture to standardize your GPU platform.
- Provide self-service tools with templates for workload onboarding.
- Define clear operational ownership between platform and workload teams.
- Monitor GPU metrics, set SLOs, and alert on key issues.
- Plan capacity, use scaling strategies, and track cost per workload.
- Implement disaster recovery with multi-region design and regular testing.
- Enforce security, governance, and supply-chain controls.
- Continuously optimize for performance, reliability, and cost.